

MEDIOtec

Curso Técnico em Informática

Fundamentos de WebDesign
Salvador Alves de Melo Júnior



Este material foi produzido pela Direção de Educação a Distância do Instituto Federal de Brasília - IFB

Reitor
Wilson Conciani

Pró-Reitor de Ensino - PREN
Adilson César de Araújo

Diretora de Educação a Distância - DEaD
Patrícia Barcelos

Coordenador Geral
Rute Nogueira de Moraes Bicalho

Coordenadora Adjunto Tecnológico
Joscélia Moreira de Azevedo

Coordenador Adjunto Administrativo
Cláudia Sabino Fernandes

Coordenador Adjunto Financeiro
Jefferson Goncalves da Silva

Coordenadora Adjunto Ensino
Jennifer de Carvalho Medeiros

Coordenadora Pedagógica
Eliziane Rodrigues de Queiroz

Orientadora de Ensino e Aprendizagem
Mércia Cristine Magalhães Pinheiro Costa

Coordenador Curso Técnico em Informática
Antongnioni Pereira de Melo

Equipe de Elaboração

Coordenadora de Produção de Material Didático Joscélia
Moreira de Azevedo

Conteudista
Salvador Alves de Melo Junior

Revisora de conteúdo
Karina Mendes Nunes Viana

Ilustrador e Produtor de vídeos animados
Márlon Cavalcanti Lima

Diagramador
Fábio Lucas Vieira

Gestora de Ambiente Virtual de Aprendizagem
Daniele Cadête de Araujo Lima

Apoio
Pró-Reitoria de Pesquisa e Inovação - PRPI

Catálogo na fonte pela Biblioteca do Instituto Federal de Educação, Ciência e
Tecnologia - Brasília



Este trabalho está licenciado com uma Licença Creative Commons -
Atribuição NãoComercial-SemDerivações 4.0 Internacional.

Sumário

Apresentação da disciplina	6
UNIDADE 1 - Arquitetura de Informação.....	9
1.1. Usabilidade.....	10
1.2. Acessibilidade.....	14
1.3. Legibilidade	20
1.4. Fundamentos e Padrões de desenho para Web	22
1.5. Interfaces para sites Web e portais.....	25
Glossário	30
UNIDADE 2 - HyperText Markup Language - HTML.....	34
2.1. Editores	38
2.2. Definições iniciais	39
2.3. Elementos básicos	42
2.3.1. Cabeçalhos.....	42
2.3.2. Parágrafos	43
2.3.3. Quebra de linha	44
2.4. Formatando textos	45
2.5. Ligando uma página a outra	48
2.6. Inserindo imagens	52
2.7. Inserindo vídeos	54
2.8. Criando tabelas	56
Glossário	63
Unidade 3 - Introdução ao CSS	67
3.1. Editores CSS.....	69
3.2. Sintaxe básica e seletores.....	71
3.3. Classes e Id's.....	73
3.3.1. Usando o id	73

3.3.2. Usando o class.....	75
3.4. Maneiras de incluir estilos	77
3.5. Uso de cores	80
3.6. Alterando os padrões de fundo	83
3.6.1. Usando um cor de fundo	83
3.6.2. Usando uma imagem de fundo	83
3.7. Textos e fontes	92
3.8. Animação com CSS.....	96
3.8.1. Transformações	97
Glossário	106
 Unidade 4 - JavaScript	 110
4.1. Editores	112
4.2. O JavaScript em um arquivo externo	116
4.3. Como mostrar informações na tela?	118
4.3.1. Usando a função alert().....	119
4.3.2. Usando a document.write ()	120
4.3.3. Usando a função getElementById() com o innerHTML	121
4.4. Elementos Básicos de Programação em JavaScript	124
4.4.1. Variáveis	124
4.4.2. Operadores Aritméticos	126
4.4.3. Uso do if e de operadores relacionais	126
4.4.4. Uso de laços de repetição	129
4.5. Criando as suas funções em JavaScript.....	133
4.6. Eventos em JavaScript.....	136
Glossário	141
Referências.....	146

apresentação

Você que adora acessar a internet, ver o seu youtuber preferido, ou acessar a sua página do Facebook, já parou para pensar como aquelas páginas são feitas?

Bom, tudo começa com uma ideia, que se junta às ideias de outras pessoas criativas. Essa criatividade deve ser traduzida em uma interface que seja fácil e agradável de aprender e de usar. A isso chamamos de usabilidade, um dos tópicos deste curso.

No final tudo precisa ser codificado, de forma que o seu navegador entenda. Essa codificação pode estar dividida em camadas. Nesse caso, podemos utilizar o HTML, o CSS e o JavaScript.

Você pode estar pensando... é muita coisa!!! Que nada, com o tempo você verá que tudo é muito mais diversão que trabalho ou estudo. Mas apenas para você ter uma ideia, vou apresentar os principais conceitos.

O HTML é responsável por apresentar o conteúdo da sua página, independente do estilo ou do comportamento da aplicação. Em outras palavras, é um texto no qual você define o conteúdo da sua página, que é interpretado pelo seu navegador, sem se importar se a cor da fonte será em vermelho ou azul.

O CSS define o estilo da sua página, sem se importar com o conteúdo. Em outras palavras, o CSS é um texto no qual você pode definir, por exemplo, que todas as tabelas terão uma fonte com a cor vermelha, independente de qual tabela será usada.

O JavaScript controla o comportamento de algum elemento da sua página, permitindo uma maior interação com o usuário, sem ter que obrigatoriamente passar pelo servidor. Então você pode, por exemplo, alertar o usuário que ele não pode deixar em branco um campo de formulário de cadastro.

Espero que você aproveite a oportunidade e construa páginas incríveis usando a técnica aqui aprendida e a sua criatividade!!





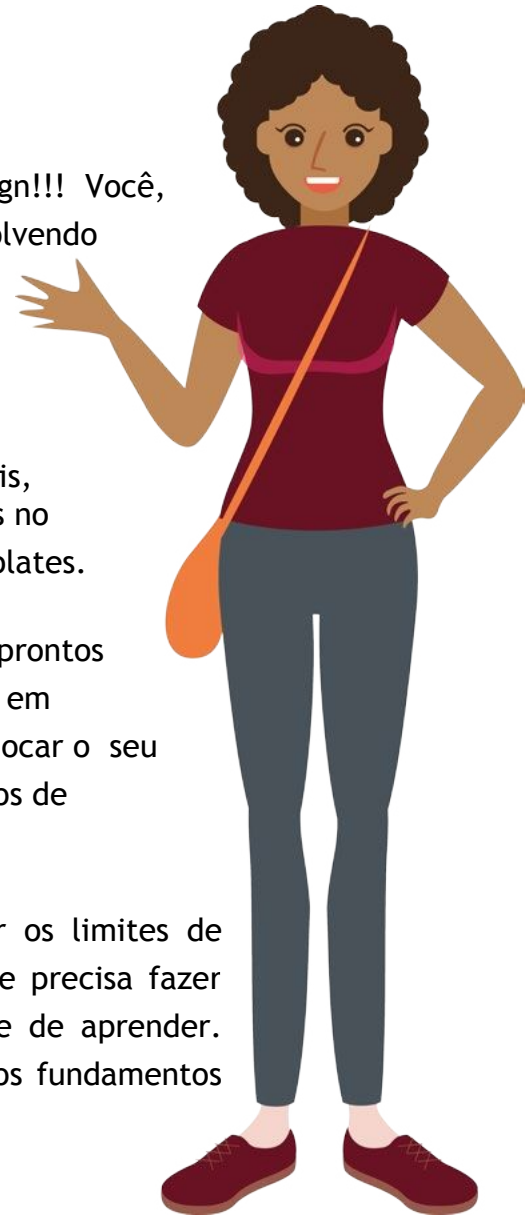
Unidade 1

Sejam bem-vindos ao curso de Fundamentos de WebDesign!!! Você, uma pessoa inteligente, irá se divertir bastante desenvolvendo páginas web, utilizando os conceitos de usabilidade, escrevendo códigos em HTML e JavaScript, formatando tudo com o CSS.

Esta disciplina não visa formar web designers profissionais, e sim, apresentar importantes conceitos, que serão úteis no desenvolvimento de sistemas e na customização de templates.

Sabe o que são templates? Neste contexto, são modelos prontos de sites, que já vêm com todo o básico que você precisa em termos de HTML, CSS e JavaScript. Assim, basta você colocar o seu conteúdo e adaptar a formatação, usando os fundamentos de WebDesign que você verá neste curso.

Ao final desta unidade, você será capaz de conhecer os limites de atuação profissional em Web Design. Identificará o que precisa fazer para desenvolver uma página que seja fácil de usar e de aprender. Conhecerá os conceitos de usabilidade, legibilidade e os fundamentos e padrões de desenho para Web.



1.1. Usabilidade

Você já parou para pensar nos equipamentos que você usa no dia a dia? Quais deles são realmente fáceis de usar? Quais deles você aprendeu a usar com facilidade?

Você pode ter certeza, todos os equipamentos que são fáceis de usar foram projetados tendo o usuário em mente.

Veja dois exemplos de um controle remoto de TV. Qual deles parece ser mais fácil de usar?



fonte da imagem 1: <https://virtualmax.com.br/image/cache/catalog/Produtos/Controle%20Semp%20Toshiba/controle-ok-500x633.png>

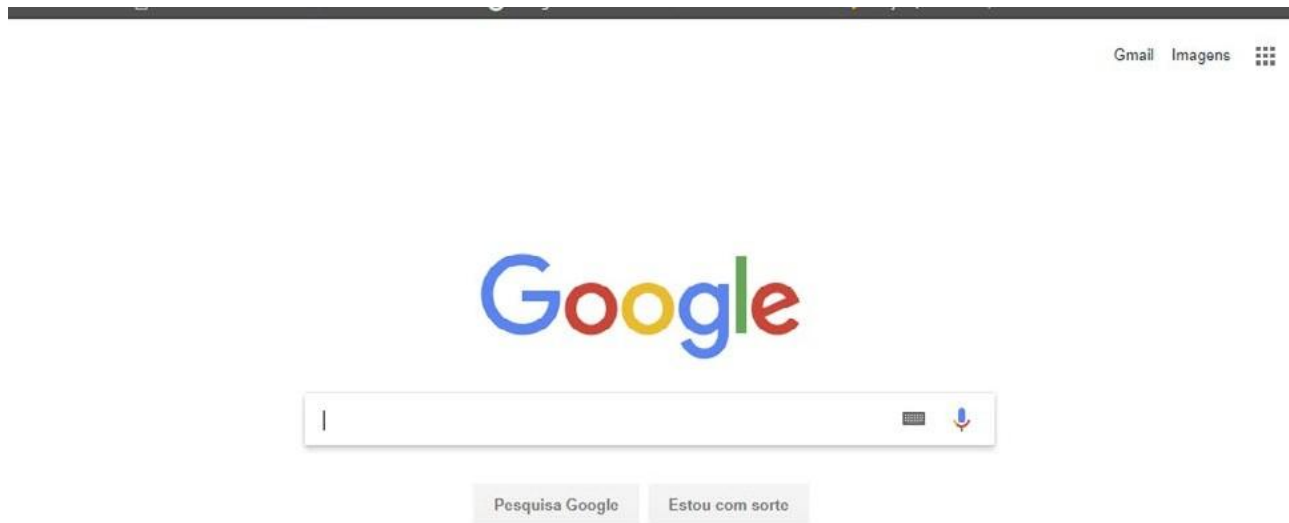


fonte da imagem 2: https://store.macstudio.com.my/wp-content/uploads/2017/06/Remote_PF-SCREEN.png

Espero como você é, já deve ter percebido que um controle remoto com menos botões, mais leve e menor, que faz coisas semelhantes, é muito mais fácil de usar e muito mais fácil de aprender a usar.

Os mesmos problemas vistos nos equipamentos podem ser identificados em sites considerados ruins na internet. Vejamos um exemplo bom e um ruim:

- Bom. O site de busca da Google tem uma finalidade principal, fazer pesquisas na internet, então sua página, simples e direta, reflete bem isso.



fonte da imagem 3: www.google.com

- Ruim. O site de Compras Governamentais têm evoluído bastante nos últimos tempos. É um importante portal governamental, tanto para os gestores, quanto para os fornecedores. Entretanto, esse portal também é responsável pela divulgação e a realização das licitações. Desafio você a localizar, em menos de 3 minutos, as licitações que foram abertas, na área de informática, nos últimos 30 dias.



fonte da imagem 4:
<https://www.comprasgovernamentais.gov.br/>

Você conseguiu perceber o problema? Sabe qual o grande problema dos exemplos negativos? Não foram projetados tendo o usuário em mente, e sim, o próprio desenvolvedor ou o gestor do ambiente. Isso se chama usabilidade.



Bora Refletir!

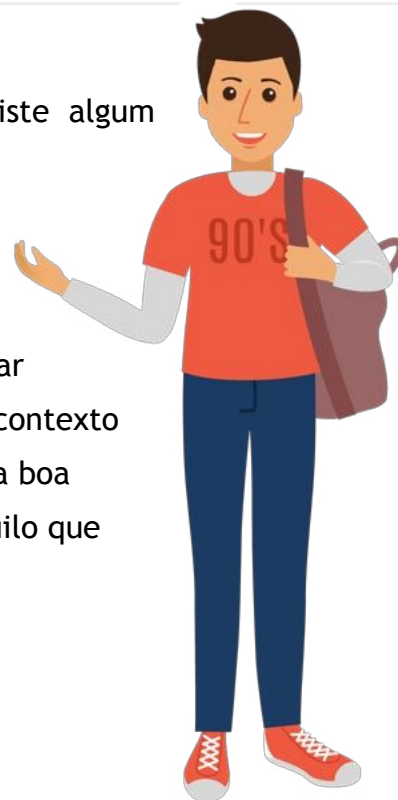
Falamos que um site tem uma boa interface, se a maioria dos usuários acha-la agradável e fácil de usar. Dissemos também que para que isso aconteça, a mesma tem que ser desenvolvida pensando no usuário. Isso

parece óbvio, mas em muitos casos, o desenvolvedor (profissional que trabalha com web designer), cria uma página imaginando o que o usuário quer ver, sem perguntar para os usuários.

Entendeu? Por isso, digo que nem sempre o óbvio é feito.

Dizer que uma página deve ter boa usabilidade é fácil. Será que existe algum documento que ajudasse o desenvolvedor a criar páginas com boa usabilidade? Sim, a Norma NBR 9241-11 .

A Norma NBR 9241-11 define que usabilidade é a medida na qual um produto ser usado por usuários específicos para alcançar objetivos específicos com eficácia, eficiência e satisfação em um contexto específico de uso. Em outras palavras, qualquer interface tem uma boa usabilidade quando é fácil de usar e de aprender e quando faz aquilo que se propõe.





Saiba mais!

A ABNT - Associação Brasileira de Normas Técnicas - é o Fórum Nacional de Normalização. As Normas Brasileiras, cujo conteúdo é de responsabilidade

dos Comitês Brasileiros (ABNT/CB) e dos Organismos de Normalização Setorial (ONS), são elaboradas por Comissões de Estudo (ABNT/CE), formadas por representantes dos setores envolvidos, delas fazendo parte: produtores, consumidores e neutros (universidades, laboratórios e outros).

Os Projetos de Norma Brasileira, elaborados no âmbito dos ABNT/CB e nos, circulam para Consulta Pública entre os associados da ABNT e demais interessados.

A NBR 9241 consiste das seguintes partes, sob o título geral de Requisitos ergonômicos para trabalho de escritório com computadores. E a parte 11 refere-se às orientações sobre usabilidade.

Fonte: <http://www.labiutil.inf.ufsc.br/cpqd-capacitacao/iso9241-11F2.doc>

1.2. Acessibilidade

Quando dissemos que uma página deve ser fácil de usar, referimo-nos também às pessoas com necessidades especiais, ou seja, deve levar em consideração os critérios de acessibilidade.

Você sabe o que é acessibilidade?

Segundo a Secretaria Especial dos Direitos da Pessoa com Deficiência, “acessibilidade é um

atributo essencial do ambiente que garante a melhoria da qualidade de vida das pessoas. Deve estar presente nos espaços, no meio físico, no transporte, na informação e comunicação, inclusive nos sistemas e tecnologias da informação e comunicação, bem como em outros serviços e instalações abertos ao público ou de uso público, tanto na cidade como no campo”. Para mais informações acesse <http://www.pessoacomdeficiencia.gov.br/app/acessibilidade-0>

Mas como fazer um site acessível? E como verificar sua acessibilidade?

Existem diversas referências de acessibilidade que podem ajudar você na construção de uma interface que seja, ao mesmo tempo, de alta usabilidade e acessível às pessoas com necessidades especiais.

O Governo Federal disponibiliza um site denominado eMAG - Modelo de Acessibilidade em Governo Eletrônico, que “tem o compromisso de ser o norteador no desenvolvimento e a adaptação de conteúdos digitais do Governo Federal, garantindo o acesso a todos”. Para mais informações acesse <http://emag.governoeletronico.gov.br/>

Mesmo que você não esteja desenvolvendo um sistema para o Governo Federal, esse site ainda assim é uma referência, pois as recomendações do eMAG “permitem que a implementação da acessibilidade digital seja conduzida de forma padronizada, de fácil implementação, coerente com as necessidades brasileiras e em conformidade com os padrões internacionais”.

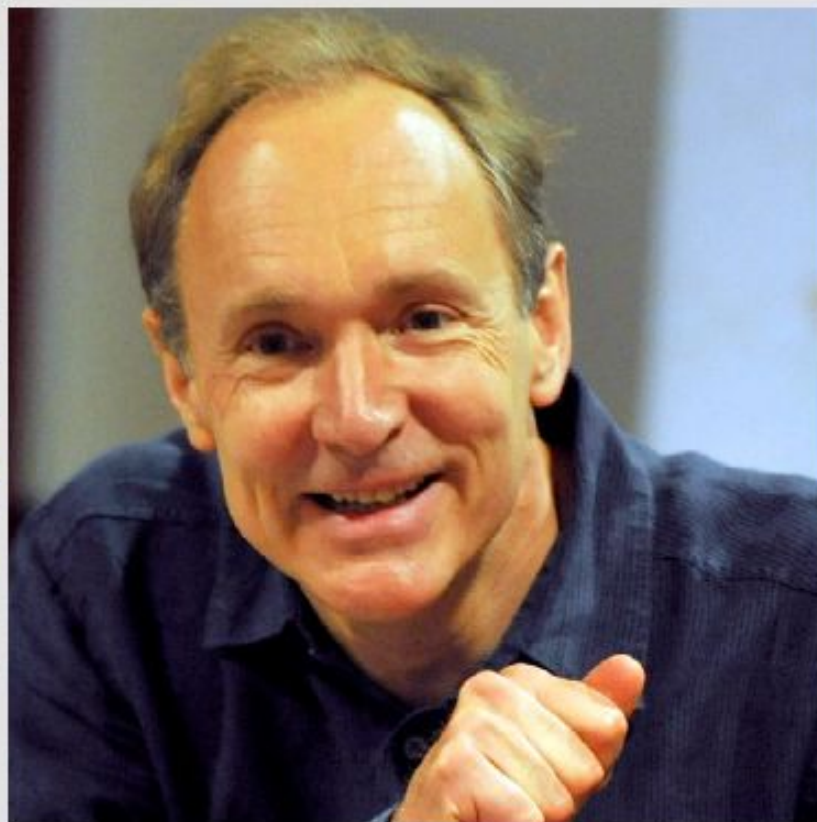
Além disso, deve-se seguir as recomendações do World Wide Web Consortium (W3C), que é uma comunidade internacional que desenvolve padrões com o objetivo de garantir o crescimento da web.



Você Sabia? (curiosidades)

Sabe quem lidera o W3C? Nada menos que o Tim Berners-Lee, o inventor da web!!!

O W3C desenvolve protocolos e diretrizes, de modo a garantir o crescimento da rede mundial de computadores (World Wide Web). Uma das maneiras de manter o crescimento é ajudar todas as pessoas a acessarem a rede mundial, inclusive as pessoas com necessidades especiais.



fonte da imagem 5: http://biografieonline.it/img/bio/Tim_Berners-Lee_4.jpg



Saiba mais!

Manual de Adaptações de Acessibilidade

O Ministério da Justiça e Cidadania, por meio da Secretaria Especial dos Direitos da Pessoa com Deficiência, elaborou um documento técnico, chamado “Manual de Adaptações de Acessibilidade”, com o objetivo de orientar o gestor do órgão federal a implementar a acessibilidade na edificação, além de fornecer informações para o preenchimento do laudo de acessibilidade e a gestão padrão.

Fonte: <http://www.pessoacomdeficiencia.gov.br/app/acessibilidade-0>





Saiba mais!

Modelo de Acessibilidade em Governo Eletrônico (eMAG)

O Ministério do Planejamento, Orçamento e Gestão, por meio da Secretaria de Logística e Tecnologia da Informação, elaborou um documento chamado “Modelo de Acessibilidade em Governo Eletrônico (eMAG)”, com o compromisso de ser o norteador no desenvolvimento e na adaptação de conteúdos digitais do Governo Federal, garantindo o acesso a todos.

As recomendações do eMAG permitem que a implementação da acessibilidade digital seja conduzida de forma padronizada, de fácil implementação, coerente com as necessidades brasileiras e em conformidade com os padrões internacionais. É importante ressaltar que o eMAG trata de uma versão especializada do documento internacional WCAG (Web Content Accessibility Guidelines: Recomendações de Acessibilidade para Conteúdo Web) voltado para o governo brasileiro, porém o eMAG não exclui qualquer boa prática de acessibilidade do WCAG.

Fonte: <https://www.governoeletronico.gov.br/documentos-e-arquivos/eMAGv31.pdf>



Saiba mais!

A W3C disponibiliza uma cartilha, que apresenta vários critérios de usabilidade que você deve observar quando for desenvolver o seu site. A referida cartilha destaca os seguintes objetivos

- O que é acessibilidade? Usar uma linguagem simples e de fácil entendimento, de modo que todos possam compreender o assunto acessibilidade;
- Quais são os grupos beneficiados com a acessibilidade? A cartilha da W3C apresenta os diferentes grupos de usuários com necessidades especiais e suas respectivas dificuldades quando tentam acessar a rede mundial;
- Como fazer um site acessível? A W3C apresenta recomendações e diretrizes que devem ser seguidas por desenvolvedores, de modo a minimizar ou eliminar as dificuldades de acesso dos grupos indicados no item anterior;
- Como avaliar um site quanto à acessibilidade? A referida cartilha apresenta orientações a respeito dos procedimentos que devem ser adotados para avaliar a acessibilidade de um site web;
- Como exigir acessibilidade? A W3C apresenta em sua cartilha algumas orientações sobre como devem proceder para cobrar a acessibilidade em sites web.

Fonte:

<http://www.w3c.br/pub/Materiais/PublicacoesW3C/cartilha-w3cbr-acessibilidade-web-fasciculo-I.html>

1.3. Legibilidade

Você já percebeu que alguns sites usam cores que dificultam a leitura? Eles usam uma combinação de cores de fundo que atrapalha a identificação das letras do texto apresentado no site, ou usam letras muito pequenas. Esses são problemas de legibilidade.

Segundo o dicionário Michaelis, legibilidade é a qualidade de legível. Em outras palavras, um site é legível quando todas as informações apresentadas na tela podem ser lidas sem dificuldade.

Como que você garante legibilidade? Comprando um óculos para o leitor?? (risos)

Você deve ficar atento, por exemplo, ao brilho do caracter, ao tamanho da fonte, ao contraste letra/ fundo, ao espaçamento entre as linhas, ao espaçamento entre os parágrafos. Veja um exemplo:



Fonte da imagem:

http://institucional.zapgrafica.com.br/wp-content/uploads/2012/05/dicas_especiais_efeitos_02.png

Quando você procura que seu site tenha um bom desempenho, deve facilitar a leitura da informação apresentada. Para isso, o desenvolvedor do site deve levar em conta duas características dos usuários: a primeira diz respeito ao cognitivo, que está relacionada com o processo de aquisição de conhecimento; a segunda está relacionada à percepção pelos sentidos.



Saiba mais!

O LabUtil da UFSC (Universidade Federal de Santa Catarina) foi o laboratório de usabilidade criado em 1995 e desativado no final de 2003, quando apoiou empresas brasileiras produtoras de software interativo que

buscavam a melhoria da usabilidade dos sistemas que produziam. Hoje, mesmo desativado, continua a disponibilizar uma lista de verificação de qualidades ergonômicas do software. O referido laboratório fez as seguintes recomendações gerais

- Títulos devem ser centralizados;
- Rótulos devem estar em letras maiúsculas;
- Cursores devem se apresentar distintos dos outros itens;
- Quando o espaço para o texto for limitado, mostrar poucas linhas longas ao invés de muitas linhas curtas;
- Exibir texto contínuo em colunas largas de, ao menos, 50 caracteres por linha;
- A justificação à direita deve ser empregada se puder ser obtida por espaçamento, desde que sejam mantidos espaçamentos proporcionais constantes entre e nas palavras e espaçamento consistente entre palavras de uma linha;
- Ao exibir um material textual, mantenha as palavras intactas, com o mínimo de hífen.

Disponível em: < <http://www.labiutil.inf.ufsc.br/index.html> >

1.4. Fundamentos e Padrões de desenho para Web

As imagens utilizadas nos sites têm evoluído bastante nas últimas décadas. Antes a gente podia exibir apenas textos. Agora podemos inserir fotos, áudios e vídeos. A velocidade de conexão evoluiu também, o que permite você ver a sua série favorita online.

Existem dezenas de formatos de imagens. Você conhece todos os principais formatos de imagens utilizados atualmente?

Segue uma tabela comparativa com as características dos principais formatos de imagens, considerando a mesma imagem, com o tamanho de 840x560.



Bora Refletir!

Você verá que existem muitas possibilidades de aplicação. Por um lado isso é bom, pois lhe dá maior flexibilidade. Por outro lado, pode te deixar em dúvida sobre qual formato usar. Fique tranquilo!

O mais importante é saber as características de cada formato, e analisar as possibilidades de acordo com as necessidades do seu projeto.

Tabela 1: formatos de imagens. (fonte: produzido pelo autor)

Formato	Principais Usos	Características	Tamanho do Arquivo
Bitmap (bmp)	Geral	• um dos formatos mais antigos e mais simples • trabalha com milhões de cores • arquivos muito grandes • não permite o efeito de fundo transparente	1361 KB
GIF	Animações e ícones	• trabalha com apenas 256 cores • permite o efeito de fundo transparente • arquivos pequenos (compactados sem perdas)	90 KB
JPEG (JPG)	Geral	• boa qualidade de imagem • trabalha com imagens de 24 bits(ou cerca de 16,8 milhões de cores) • arquivos pequenos (compactados com perdas¹)	130 KB
PNG	Animações e ícones	• trabalha com milhões de cores • permite o efeito de fundo transparente • arquivos pequenos (compactados sem perdas) • exibe detalhes com mais nitidez que a JPEG, pois sua compactação é sem perdas • utilização apoiada pela W3C	384 KB

RAW	Uso profissional de fotografia	• guarda todos os dados da imagem, tal como captada pelo sensor da câmara fotográfica, sem aplicação de efeitos ou ajustes	Muito grande (tamanho depende da máquina fotográfica)
SVG	Imagens estáticas e animadas	• imagens vetoriais • formato aberto • permite o efeito de fundo transparente • podem ser ampliadas ou reduzidas sem causar perda de qualidade	503 KB
TIFF	imagens médicas e industriais	• grande quantidade de cores • excelente qualidade de imagem • suporta o uso de camadas • permite o efeito de fundo transparente	428 KB
WebP	Geral	• formato de imagem desenvolvida pela Google • tamanho reduzido • rápido carregamento • boa qualidade de imagem • concorrente do JPEG	28 KB

¹ JPEG: realiza a compactação da imagem original, com perdas de informação. Porém, se efetuada dentro de certos limites, as perdas não são perceptíveis ao olho humano.

1.5. Interfaces para sites Web e portais

De forma análoga ao que foi dito anteriormente, a pesquisadora Camila Brandão da PUC-SP, destaca que apesar da importância de qualquer interface contemplar as necessidades e desejos dos usuários, existem muitos produtos digitais de difícil uso e pouco prazerosos para os usuários, representando uma baixa usabilidade. A referida pesquisadora referencia também os três principais problemas de interfaces mal idealizadas, a saber

- Ignorância em relação aos usuários: não sabe ao certo quais são as características do usuário da interface e nem o que a mesma deveria ter para deixá-los felizes;
- O conflito de interesse entre as necessidades dos usuários e as prioridades de construção do produto digital: é comum ter um conflito entre o que pode ser feito, com o tempo disponível, com a equipe contratada e da forma mais fácil;
- Falta de processo para entender as necessidades dos usuários: nem sempre são estabelecidos processos para realmente entender o que o usuário final da interface precisa.



Saiba mais!

Como podemos criar sites que sejam agradáveis e fáceis de usar?

Abordaremos duas propostas básicas

- Design Thinking: a ideia principal é pensar numa solução utilizando diferentes pontos de vista, de forma colaborativa e coletiva, envolvendo todas as pessoas relacionadas com o projeto, desde os patrocinadores até os usuários finais. E tudo fica melhor se envolver pessoas com vivências e formações diferentes, em equipes multidisciplinares. Assim, teremos diferentes modos de ver os problemas, testando diversas ideias e encontrando possíveis soluções.

Leia este material da InovaGov sobre “Design Thinkin“. Disponível em:

<http://nova.gov.br/wp-content/uploads/2017/12/ebookconfiancacriativa.pdf>.

- User Experience (UX): o ponto focal dessa abordagem é como tornar a experiência do usuário da interface mais fácil e útil. Ou seja, focar na usabilidade, com a apresentação do conteúdo disposta de forma lógica, do ponto de vista da experiência do usuário.

Assista a este vídeo da Canaltech Startups e saiba um pouco mais sobre “O que é UX (User Experience)? E o que NÃO é UX?”. Disponível em: < https://youtu.be/2hfu_F4cDRQ >.



Questões de autoaprendizagem

Vamos pensar e praticar...

Vamos supor que você seja ligado no mundo da internet. Nesse caso, talvez seja fácil imaginar uma nova interface para um novo tablet, que acessaria sua série favorita do Netflix (Stranger Things??) ou os vídeos do seu Youtuber preferido (Whindersson Nunes??). Mas vamos tirar você da sua zona de conforto.

Gostaria que você imaginasse uma nova interface do tablet para um público específico, o idoso. Imagine as funcionalidades que esse tablet deveria ter e faça um esboço da tela inicial.

Solução...

Entendeu que se colocar no lugar do outro, no caso, o usuário, é necessário e deve ser feito com cuidado? Então, você precisa ouvir o usuário, conhecer o perfil da pessoa que irá utilizar, saber onde serão utilizados, conhecer as atividades que as pessoas estão realizando quando interagem com o produto (que pode ser um site) e imaginar como você poderia fazer para otimizar a interação do usuário com o sistema.

Uma proposta de solução seria esta:

TabSênior

É Tablet, É Smartphone
e é fácil de mexer!



by



fonte da imagem 7: <https://canaltech.com.br/tablet/conheca-o-primeiro-tablet-para-idosos-do-brasil-97290/>



Síntese final do módulo (Bora rever!)

Nesta lição a gente viu o que seria um bom projeto de interface. Vimos que um bom projeto deve ter o usuário como foco central. Parece óbvio, mas quantas vezes você já entrou em um site ruim, difícil de usar e chato

para aprender.

Conhecemos também um pouco sobre usabilidade, uma ciência que tem evoluído continuamente. Essa usabilidade envolve também aspectos de acessibilidade, garantindo o acesso às pessoas com necessidades especiais. E envolve também critérios de legibilidade. Já viu quantas interfaces são difíceis de ler?

A gente estudou alguns padrões de formato de imagens. Talvez você nunca tenha usado alguns desses formatos, mas vai usar. Pode ter certeza!

Por fim, identificamos algumas estratégias para elaborar interfaces agradáveis e fáceis de usar.



Sites indicados

CARTILHA: ACESSIBILIDADE NA WEB - W3C BRASIL. Disponível em: <http://www.w3c.br/pub/Materiais/PublicacoesW3C/cartilha-w3cbr-acessibilidade-web-fasciculo-1.html> >. Acesso em: 14 jan. 2018.

Laboratório de Utilizabilidade da Informática - Home Page. Disponível em: <http://www.labiutil.inf.ufsc.br/index.html> >. Acesso em: 14 jan. 2018.

Modelo de editoração de Normas ABNT - LabIUtil. Disponível em: <http://www.labiutil.inf.ufsc.br/cpqd-capacitacao/iso9241-11F2.doc> >. Acesso em: 14 jan. 2018.



Glossário

- Acessibilidade

É um atributo essencial do ambiente que garante a melhoria da qualidade de vida das pessoas. Deve estar presente nos espaços, no meio físico, no transporte, na informação e comunicação, inclusive nos sistemas e tecnologias da informação e comunicação, bem como em outros serviços e instalações abertos ao público ou de uso público, tanto na cidade como no campo.

Fonte: <http://www.pessoacomdeficiencia.gov.br/app/acessibilidade-0>

- Legibilidade

Diz respeito às características lexicais das informações apresentadas na tela que possam dificultar ou facilitar a leitura desta informação (brilho do caracter, contraste letra/fundo, tamanho da fonte, espaçamento entre palavras, espaçamento entre linhas, espaçamento de parágrafos, comprimento da linha, etc.).

Fonte: http://www.labiutil.inf.ufsc.br/CriteriosErgonomicos/LabUtil2003-Crit/140legib_A.html

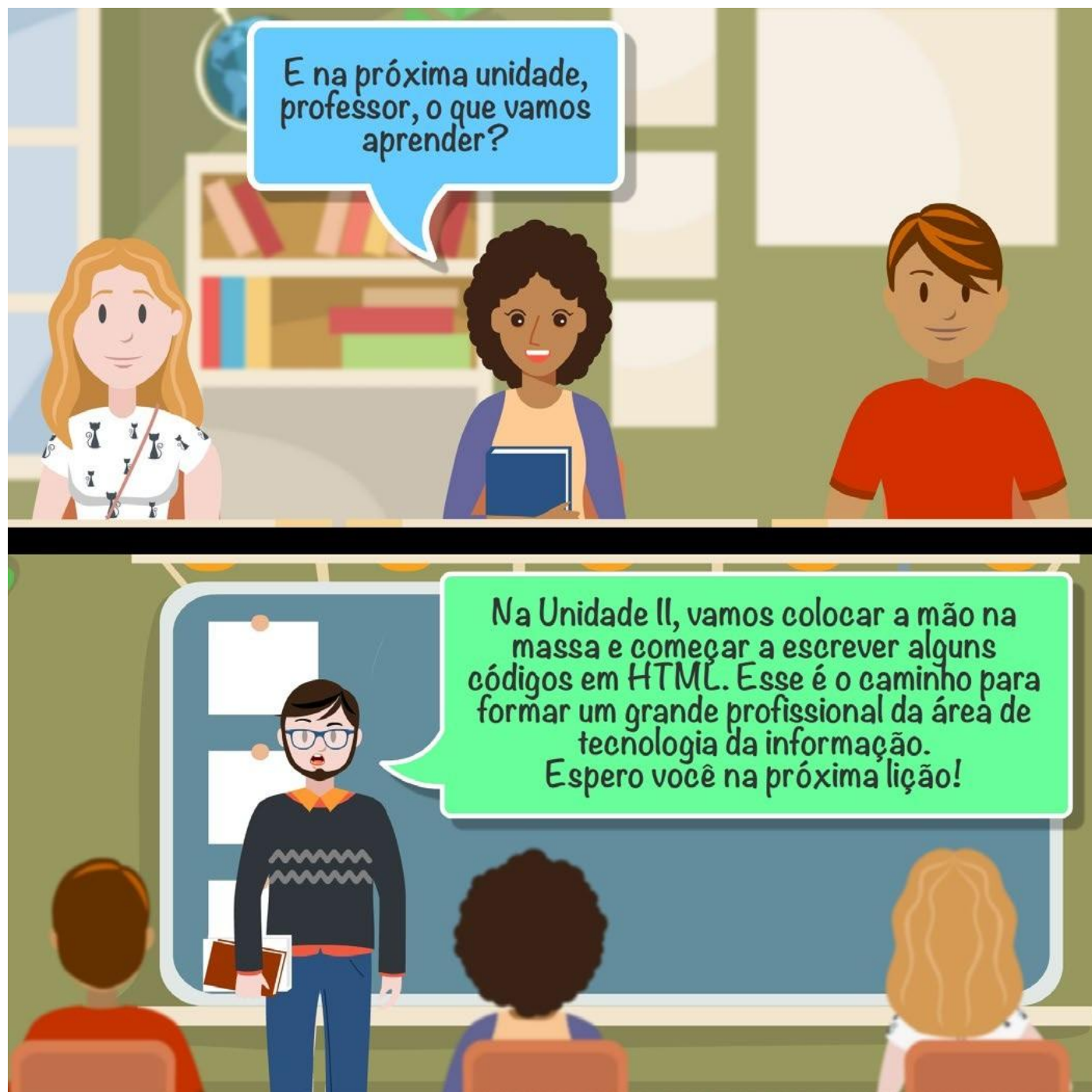
- Usabilidade

Estudo ou a aplicação de técnicas que proporcionem a facilidade de uso de um dado objeto, no caso, um sítio.

Fonte: <http://epwg.governoeletronico.gov.br/cartilha-usabilidade#s1.1>

Neste capítulo aprendemos os conceitos e teorias utilizadas no design para web.
Consegue perceber a necessidade de ouvir o usuário durante o processo de criação de uma interface?
Aproveite para consultar o glossário se tiver dúvidas sobre algum termo. Vamos partir para algumas questões de aprendizagem e para a revisão final da unidade.
Não se esqueça de olhar os sites e vídeos indicados que irão ampliar os seus horizontes de conhecimentos sobre o tema da unidade.







Curso Técnico em Informática

Fundamentos de Web Design

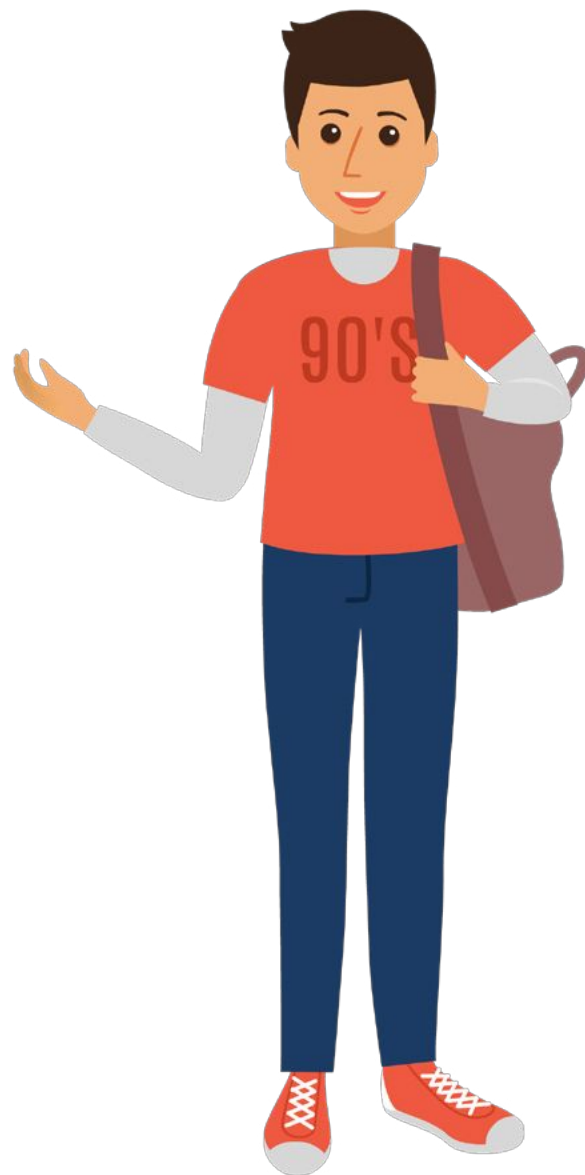
Unidade 2

HyperText Markup Language - HTML

Na primeira unidade, você viu que devemos desenvolver interfaces que sejam agradáveis e fáceis de usar, que podem ser um site, uma página web. Para criar essa página, você deve usar uma linguagem específica, que permita navegador a reconheça.

Assim, nesta unidade aprenderemos sobre HTML, uma abreviação para HyperText Markup Language, que em português podemos traduzir para Linguagem de Marcação de Hipertexto. Essa linguagem é utilizada para a construção de páginas web.

Ao final desta unidade, você aprenderá os principais comandos do HTML, criando páginas estáticas. Você não será um desenvolver HTML profissional apenas com este curso, pois o objetivo não é esse, e sim fornecer o conhecimento necessário para customizar templates e criar páginas mais simples, o conhecimento básico que todo desenvolvedor web deve ter.





Você Sabia? (curiosidades)

De onde surgiu essa ideia de internet? Será que um dia todo mundo acordou e passou a usar o WhatsApp?

Esse ao homem ao lado é um físico britânico, cientista da computação e professor do MIT chamado Timothy John Berners-Lee, que criou o HTML com o objetivo inicial de compartilhar suas pesquisas com seus colegas. Naquela época a página da internet não ia além de textos, com pouca interatividade, sem imagens ou vídeos. Mas foi o início de uma tremenda revolução no modo como nos comunicamos.

Será que você será o próximo Berners-Lee? Só o tempo dirá.



fonte da imagem 1: <https://www.csaspeakersindia.in/media/pictures/profile/TIMBER.png>

Quando todo mundo começou a usar o HTML, as coisas estavam um pouco confusas, sem um padrão. Então, em 1990 o HTML foi formalizado como uma linguagem, seguindo determinados padrões. Nessa mesma linha, em 1994, Berners-Lee criou o famoso World Wide Web Consortium, também conhecido com W3C, que é um consórcio com cerca de 400 membros, cujo objetivo principal é a padronização da World Wide Web, também conhecida como rede mundial de computadores ou simplesmente web.

Segundo a W3C

(https://www.w3.org/standards/techs/html#w3c_all),

sempre se buscou novos recursos para o HTML, que continuam a ser introduzidos para ajudar os autores de aplicativos da Web. Desse modo, novos elementos continuam a ser introduzidos com base na pesquisa sobre as práticas de autoria prevalentes.

Com isso, com todas as contribuições, o HTML tem evoluído muito ao longo dos anos, acompanhando as novas necessidades e a evolução das tecnologias. Resumidamente, apresentamos as principais versões do HTML:

Versão	Ano
HTML 5.2	2017
HTML 5.1	2016 e 2017 (segunda edição)
HTML 5	2014
XHTML	2000
HTML 4.01	1999
HTML 3.2	1997
HTML 2.0	1995
HTML	1991

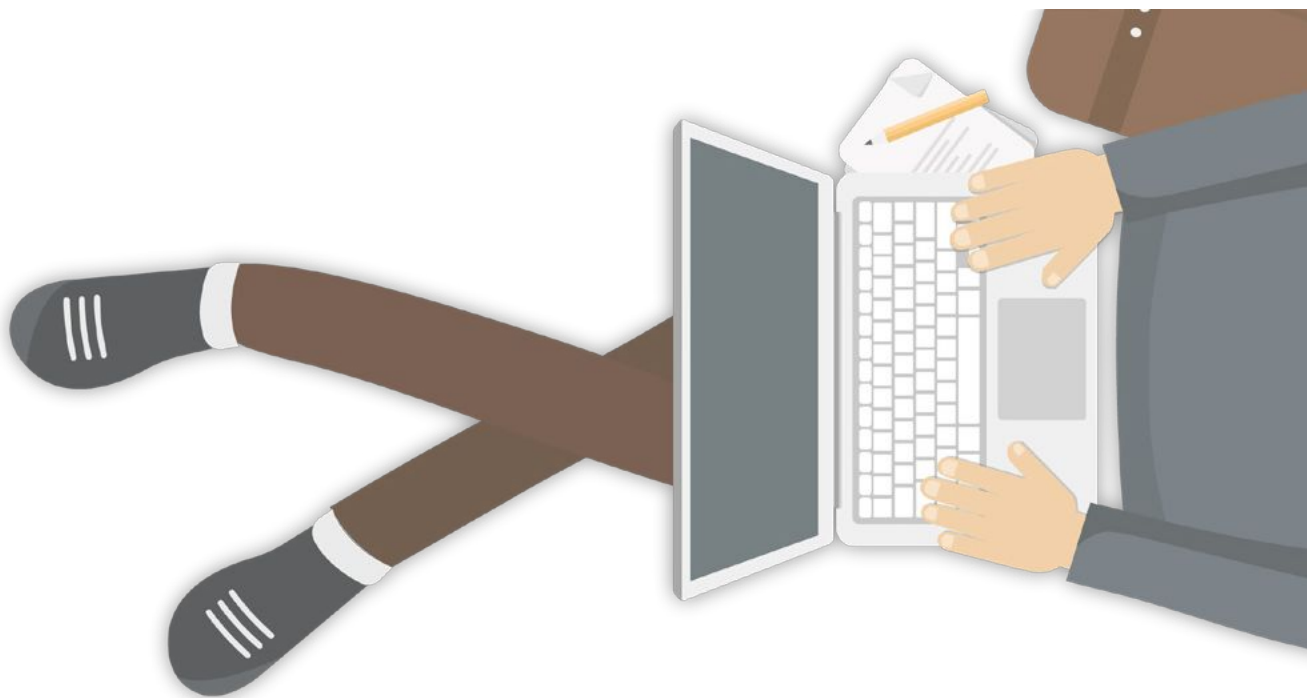
Tabela 1: versões do HTML (Fonte: elaborado pelo autor).



Saiba mais!

O Consórcio World Wide Web (W3C) é um consórcio internacional, no qual organizações filiadas, uma equipe em tempo integral e o público trabalham juntos para desenvolver padrões para a Web. Liderado pelo inventor da web, Tim Berners-Lee e o CEO Jeffrey Jaffe, o W3C tem como missão conduzir a World Wide Web para que atinja todo seu potencial, desenvolvendo protocolos e diretrizes que garantam seu crescimento de longo prazo.

Fonte: <http://www.w3c.br/Home/WebHome>



2.1. Editores

A gente já falou sobre a história e a importância do HTML. Agora vamos colocar em prática, vamos desenvolver um código HTML, vamos criar uma página web.

Por onde você deve começar? Afinal do que você precisa para criar um código HTML?

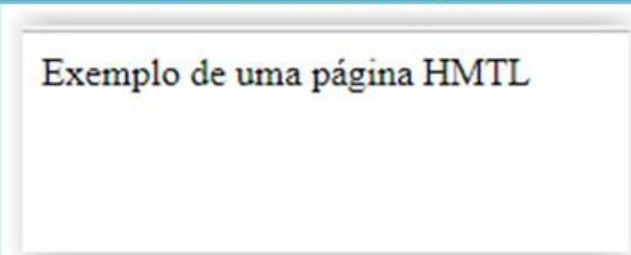
A rigor, basta ter um editor de texto simples (bloco de notas) para criar um código HTML (neste caso você deve salvar como um arquivo .html) e um navegador para visualizar o resultado do referido código. Existem dezenas de ambientes de desenvolvimento. Vou apresentar para você os principais.

Nome	Breve descrição	Onde encontrar?
Adobe Dreamweaver	Este é provavelmente o mais famoso programa para desenvolvimento de páginas web.	http://www.adobe.com/br/products/dreamweaver.html
Notepad++	É um editor de código-fonte gratuito, regido pela Licença GPL ² .	https://notepad-plus-plus.org/
Sublime Text	Semelhante ao Notepad++, porém com uma interface melhorada.	https://www.sublimetext.com/
Brackets	Um editor de códigos leve e limpo com ótimas funções de apoio.	brackets.io/
NetBeans	É um ambiente de desenvolvimento integrado gratuito e de código aberto para desenvolvedores de software de várias linguagens.	https://netbeans.org/downloads/

Testando os exemplos

Para entender a teoria sobre HTML, é muito importante testar os exemplos e praticar. Vou apresentar como fazer esses testes, de uma maneira muito simples. Para isso, peço que você siga estes passos

1. Abra o aplicativo chamado “bloco de notas”
2. Escreva o código a seguir
3. Salve o arquivo com o nome exemplo.html
4. Dê dois cliques em cima do arquivo exemplo.html

Código HTML	Resultado no Navegador
<pre><html> <body> Exemplo de uma página HTML </body> </html></pre>	

2.2. Definições iniciais

Agora que você já sabe que precisa praticar, vamos aprender alguns conceitos iniciais. Por exemplo, como um navegador sabe o que é um texto simples e o que é um código HTML?

Os códigos HTML ficam entre as marcações (ou também chamadas tags) `<` e `>`, que geralmente são apresentadas aos pares, ou seja, `<marcação>` e `</marcação>`. Exemplo:

```
<html> ... </html>
```

Entretanto, há algumas exceções para o funcionamento aos pares, que a gente vai explicar à medida que elas forem aparecendo, mas já podemos citar a marcação que indica um parágrafo `<p>` que não necessita, obrigatoriamente, de um `</p>`.



Você Sabia? (curiosidades)

O navegador é um aplicativo que interpreta códigos em HTML, permitindo a interação do usuário com o conteúdo apresentado em uma página web. Esses aplicativos têm evoluído muito, acompanhando a evolução das tecnologias.

Hoje existem diversos navegadores, desenvolvidos por diferentes empresas. Por exemplo:

-  Chrome da Google
-  Firefox da Mozilla
-  Internet Explorer da Microsoft
-  Safari da Apple

Existe um padrão estabelecido pela W3C e determinadas exigências de mercado, mas não existe uma lei que obrigue todos os desenvolvedores de navegadores a seguir tais padrões e exigências. E se o navegador não entende uma determinada marcação, ele simplesmente a ignora. Portanto, é importante realizar um cross-browser, ou seja, testar uma determinada página em diferentes navegadores.

Existem diversas soluções online que podem lhe ajudar a realizar esses testes. Veja alguns exemplos:

- Browsera. Disponível em: <<https://turbo.net/browsers>>
- Browserling. Disponível em: <<https://www.browserling.com/>>
- Browsershots. Disponível em: <<http://browsershots.org/>>
- IE NetRenderer. Disponível em: <<http://netrenderer.com/index.php>>
- Viewlike.us. Disponível em: <<https://www.viewlike.us/>>>

Além da marcação <html> existem outras, que podem ser consideradas as marcações mínimas de uma página. Preste atenção!

<code><!DOCTYPE html></code>	• define este documento como HTML5
<code><html></code>	• elemento raiz de uma página HTML
<code><title></code> Exemplo	• especifica um título para o documento
<code></title></code>	
<code><head></code>	• contém meta-informações sobre o documento
<code><meta charset="UTF-8"></code>	• define o conjunto de caracteres a ser usado
<code></head></code>	
<code><body></code> Minha Página	• local do conteúdo da página visível
<code></body></code>	
<code></html></code>	



Você Sabia? (curiosidades)

Apesar de existir um padrão, na hora em que você está digitando seu código HTML, pode acontecer de você trocar alguma letra maiúscula pela equivalente minúscula e vice-versa. Não esquite a cabeça com isso!!! As marcações HTML não são “case sensitive”, não diferenciam letras maiúsculas de minúsculas. Em outras palavras, <HTML> significa o mesmo que <html>.

2.3. Elementos básicos

Vamos agora estudar os elementos básicos de uma página HTML. São várias marcações que aparecem com maior frequência.

2.3.1. Cabeçalhos

No contexto do HTML, você utiliza cabeçalhos para definir um destaque para um determinado texto e podem ser definidos 6 níveis de cabeçalhos. Segue a codificação básica:

`<Hnúmero>...</Hnúmero>`

Exemplo:

Código HTML	Resultado no Navegador
<pre><!DOCTYPE html> <html> <title>Título da página</title> <head> <meta charset="UTF-8"> </head> <body> <h1> destaque </h1> <h2> destaque </h2> <h3> destaque </h3> <h4> destaque </h4> <h5> destaque </h5> <h6> destaque </h6> </body> </html></pre>	

2.3.2. Parágrafos

Agora a gente vai estudar como são inseridas quebras de parágrafo no HTML.

O texto de uma página web geralmente é dividido em parágrafos, como normalmente a gente faz fora da web. Entretanto, os navegadores não reconhecem as quebras de parágrafos que você coloca no código HTML.

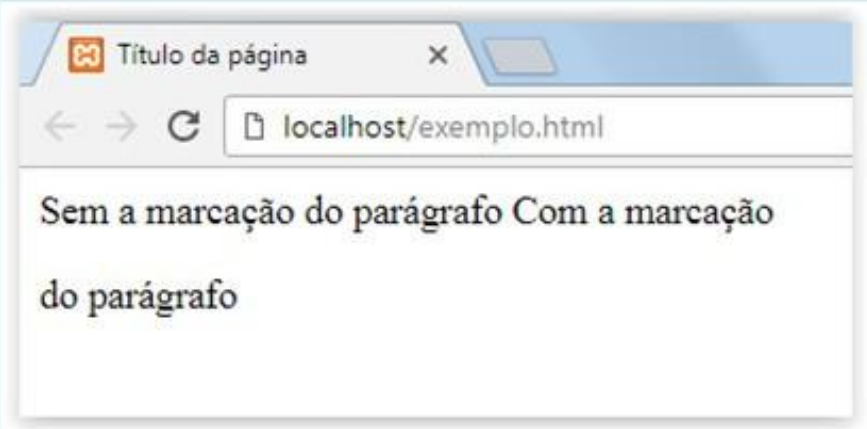
Os navegadores inserem uma quebra de linha ou a marcação de um novo parágrafo apenas quando o texto chega no final da janela. Para inserir a quebra de um novo parágrafo manualmente, você precisa inserir a marcação `<p>`, preferencialmente seguida da marcação `</p>`.

Exemplo:

Código HTML

```
<!DOCTYPE html>
<html>
  <title>Título da página</title>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    Sem a marcação do parágrafo
    Com a marcação <p> do parágrafo
  </body>
</html>
```

Resultado no Navegador



2.3.3. Quebra de linha



Bora Refletir!

O espaço entre linhas não pode ser exagerado. Talvez você não queira colocar o espaço extra as linhas, que aparece sempre que você usa a marcação `<p>`.

O que fazer?

Neste caso, você precisa usar a marcação `<br>`, que quebra a linha, sem acrescentar um espaço extra entre as linhas.

Exemplo:

Código HTML

```
<!DOCTYPE html>
<html>
  <title>Título da página</title>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    Com a marcação <p> do parágrafo
    Com a marcação <br> de quebra de linha
  </body>
</html>
```

Resultado no Navegador

Título da página

localhost/exemplo.html

Com a marcação
do parágrafo Com a marcação
de quebra de linha

2.4. Formatando textos

Tão importante quanto o texto que você apresenta na página web é a formatação e o destaque com que você apresenta o referido texto.

A linguagem HTML permite inserir diversas formatações em um texto qualquer. As principais marcações relacionadas à formatação são as seguintes:

Marcação	Função
<code></code>	Colocar o texto em negrito.
<code></code>	Visualmente faz o mesmo que a marcação <code>b</code> , basicamente é apenas uma recomendação mais nova.
<code><i></code>	Coloca o texto em itálico.
<code></code>	Visualmente faz o mesmo que a marcação <code>i</code> , basicamente é apenas uma recomendação mais nova.
<code><mark></code>	Coloca marcado com um uma cor de destaque.
<code><small></code>	Coloca o texto em destaque, diminuindo seu tamanho em relação ao tamanho padrão.
<code></code>	Coloca um traço sobre o texto para marcar que o referido trecho foi excluído do texto original.
<code><ins></code>	Coloca um traço sob o texto para marcar que o referido trecho foi incluído no texto original.
<code><sub></code>	Coloca o texto subescrito.
<code><sup></code>	Coloca o texto sobreescrito.

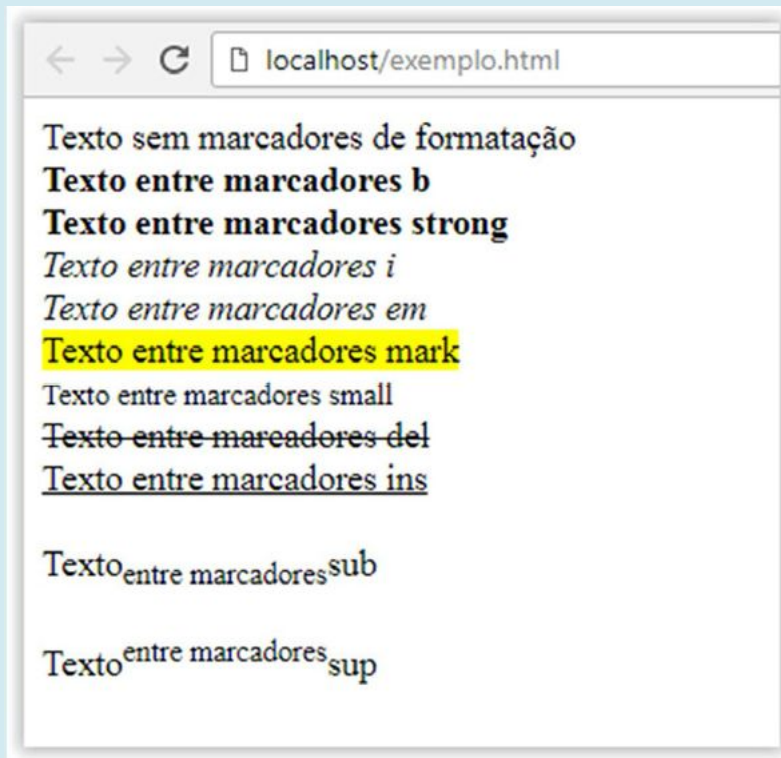
Tabela 2: formatações básicas de texto (fonte: elaborada pelo autor)

Exemplo:

Código HTML

```
<!DOCTYPE html>
<html>
  <title>Título da página</title>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    Texto sem marcadores de formatação<br>
    <b>Texto entre marcadores b</b><br>
    <strong>Texto entre marcadores strong</strong><br>
    <i>Texto entre marcadores i</i><br>
    <em>Texto entre marcadores em</em><br>
    <mark>Texto entre marcadores mark</mark><br>
    <small>Texto entre marcadores small</small><br>
    <del>Texto entre marcadores del</del><br>
    <ins>Texto entre marcadores ins</ins><br><br>
    Texto<sub>entre marcadores</sub>sub<br><br>
    Texto<sup>entre marcadores</sup>sup
  </body>
</html>
```


Resultado no Navegador



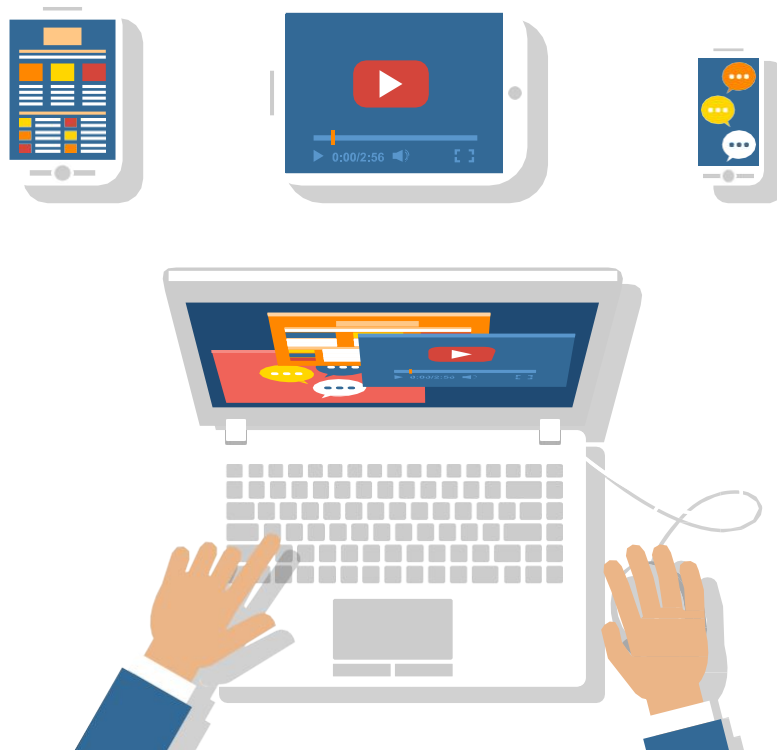
2.5. Ligando uma página a outra

Imagine algo comigo! Você está na página do Youtube, vê um vídeo legal, talvez do seu Youtuber preferido e clica no link do vídeo. Então, a página com o vídeo que você selecionou aparece no navegador.

Essa é uma das grande vantagens do HTML: a possibilidade de você poder navegar pelas diferentes páginas, sem ter que seguir uma ordem pré-definida e sim o interesse do usuário.

Mas como fazer isso?

Usando a marcação de âncora.



Veja o passo a passo para criar um link para uma outra página:

1. Utilizar a marcação <a>
2. Utilizar o atributo href para indicar qual página será “linkada”
3. Definir o texto ou imagem que servirá de link
4. Fechar com a marcação

Exemplo:

Página 1	Página 2
<pre><!DOCTYPE html> <html> <title>Página 1</title> <head> <meta charset="UTF-8"> </head> <body> <h1>Página 1</h1> Link para a página 2 </body> </html></pre>	<pre><!DOCTYPE html> <html> <title>Página 2</title> <head> <meta charset="UTF-8"> </head> <body> <h1>Página 2</h1> Link para a página 1 </body> </html></pre>



Você Sabia? (curiosidades)

O chamado Hypertext Transfer Protocol (ou na sigla HTTP, que em tradução livre do inglês significa Protocolo de Transferência de Hipertexto) é um protocolo de comunicação (na camada de aplicação segundo o Modelo OSI) utilizado para sistemas de informação de hipermídia distribuídos e colaborativos.

(Fonte:

<https://www.portaleducacao.com.br/conteudo/artigos/idiomas/o-protocolo-http/20408>)

E, se ao invés de direcionar para uma outra página do seu próprio portal, você quisesse criar um link para um site externo? Por exemplo, criar para a página do Youtube? Para isso, basta apresentar o endereço completo, inclusive com o respectivo protocolo, geralmente http.

Exemplo:

Página 1

```
<!DOCTYPE html>
<html>
  <title>Página 2</title>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    Acesse o youtube clicando
    <a href="https://www.youtube.com/">
      aqui
    </a>
  </body>
</html>
```

Resultado no Navegador

Antes de clicar

Depois de clicar



2.6. Inserindo imagens

Mas uma página não deveria ser feita apenas de textos, correto? E uma imagem pode dizer mais que 1000 palavras. Para inserir uma imagem em uma página HTML basta utilizar a marcação `<img>` e alguns atributos, a saber:

- `src`: define o nome do arquivo da imagem;
- `alt`: define o texto que será apresentado, no lugar da imagem, quando a imagem não pode ser exibida;
- `width`: define a largura da imagem (em pixels);
- `height`: define a altura da imagem (em pixels).

Veja o exemplo a seguir:

Usando uma imagem local	Usando o link para uma imagem
<pre> <!DOCTYPE html> <html> <title>Página 2</title> <head> <meta charset="UTF-8"> </head> <body> <h1>Dragon Ball Z</h1> </body> </html> </pre>	<pre> <!DOCTYPE html> <html> <title>Página 2</title> <head> <meta charset="UTF-8"> </head> <body> <h1>Dragon Ball Z</h1> </body> </html> </pre>
Resultado no Navegador	Resultado no Navegador
	

2.7. Inserindo vídeos

Por falar em Youtube, esse é um dos maiores sites de vídeos do mundo, onde todos podem se divertir,

aprender e, muitas vezes, se chocar.

E se você quiser inserir um vídeo na sua página? É fácil. Vamos aprender a fazer?

Com o HTML 5 a inserção de vídeos em uma página web ficou muito mais fácil. Você precisa utilizar

basicamente duas marcações:

- `<video>`: que define que você irá inserir um vídeo;
- `<source>`: onde você especifica o formato do vídeo;
- `<iframe>`: use para incorporar um outro documento dentro do HTML.

Junto com essas marcações, você pode utilizar alguns atributos que podem ser muito úteis, a saber:

- `width`: você especifica a largura da janela do vídeo;
- `height`: você especifica a altura da janela do vídeo;
- `controls`: você permite que os controles de reprodução, pausa e volume sejam visualizados;
- `autoplay`: você define que o vídeo seja inicializado automaticamente;
- `src`: você especifica onde encontrar o arquivo do vídeo, seja numa pasta local ou na web;
- `allowfullscreen`: você especifica que o vídeo pode ser visto em tela cheia;
- `type`: você especifica o tipo de arquivo. Atualmente, os principais tipos de arquivos são os

seguintes:

- o `mp4`: formato padrão de compactação de áudio e vídeo;
- o `ogg`: um formato orientado a stream, ou seja, não precisa fazer o download de todo o

Usando um vídeo local	Usando o link para um vídeo na web
<pre> <!DOCTYPE html> <html> <title>Exemplo vídeo</title> <head> <meta charset="UTF-8"> </head> <body> <video width="400" height="300" allowfullscreen controls autoplay> <source src="Wanna_cry_virus_in_action.mp4"> </video> </body> </html> </pre>	<pre> <!DOCTYPE html> <html> <title>Exemplo vídeo</title> <head> <meta charset="UTF-8"> </head> <body> <iframe width="560" height="315" src="https://www.youtube.com/embed/kJ QP7kiw5Fk" frameborder="0" allowfullscreen> </iframe> </body> </html> </pre>
Resultado no Navegador	Resultado no Navegador
	

2.8. Criando tabelas

E se você quiser apresentar alguns dados em página de forma organizada e ordenada, divididos em colunas e linhas? Por exemplo, um site que apresenta os resultados do Brasileirão. Nesse caso você poderia apresentar os dados na forma de uma tabela do HTML. Agora você vai aprender a criar tabelas.

Para criar uma tabela, você precisa usar pelo menos as seguintes marcações:

- `<table>`: define onde começa e termina uma tabela;
- `<tr>`: define uma nova linha da tabela;
- `<th>`: define um célula especial, do tipo cabeçalho, com destaque;
- `<td>`: define uma célula de dados.

Além disso você pode utilizar os seguintes atributos:

- `border`: permite incluir as linhas de borda;
- `align`: permite definir o alinhamento ao centro (center), à esquerda (left) ou à direita (right);
- `rowspan`: permite mesclar duas linhas;
- `colspan`: permite mesclar duas colunas.

Exemplo:

Tabela em HTML

```
<!DOCTYPE html>
<html>
  <title>Exemplo vídeo</title>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <table border="1">
      <tr>
        <td rowspan="2">
          
        </td>
        <td align="left">Hermione Granger</td>
        <td align="right">Ronald Weasley</td>
      </tr>
      <tr>
        <td colspan="2" align="center">Harry Potter</td>
      </tr>
    </table>
  </body>
</html>
```

Resultado no Navegador



Questões de autoaprendizagem

Vamos pensar e praticar...

Sei que vocês têm seus programas e séries favoritas, seus sites favoritos, as coisas com as quais você consegue passar horas vendo e se divertindo. Vamos contar isso para todo mundo?

Crie uma página reproduzindo a página a seguir. Seja criativo e pode mudar o conteúdo e os temas, mas não a estrutura. Tenho certeza de que você consegue, pois é uma pessoa muito inteligente. Atenção: as imagens são apenas ilustrativas.

Coisas que eu gosto

Meu desenho favorito



Meu game preferido



Meu youtuber preferido



Solução...

Para resolver essa questão, você deveria primeiro pensar na estrutura da tabela e na forma que você quer apresentar. Depois, pesquisar algumas imagens que são interessantes para você. Por fim, deveria escrever um código semelhante a este:

```
<!DOCTYPE html>
<html>
  <title>
    Exemplo
  </title>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <table border>
      <tr>
        <th colspan="2">Coisas que eu gosto</th>
      </tr>
      <tr>
        <td>Meu desenho favorito</td>
        <td>
          
          </td>
        </tr>
      <tr>
        <td>Meu game preferido</td>
        <td>
```



```

        
    </tr>
    <tr>
        <td>Meu youtuber preferido</td>
        <td>
            
        </tr>
    </tr>
</table>
</body>
</html>

```



Síntese final do módulo (Bora rever!)

Nesta lição a gente viu o que é e qual a importância do HTML. Conhecemos alguns ambientes de desenvolvimento, entendemos alguns conceitos, aprendemos as marcações básicas, conhecemos algumas formatações de

texto, aprendemos como ligar uma página a outra, inserimos imagens e vídeos e, por fim,

criamos tabelas, tudo isso com o HTML.

Agora você está preparado para a próxima lição, em que aprenderá a formatar a sua página, usando uma folha de estilos.



Sites indicados

HTML5 Tutorial. Disponível em: [<https://www.w3schools.com/html/>](https://www.w3schools.com/html/)

Acesso em: 14 jan. 2018.

W3C Brasil. Disponível em: [<http://www.w3c.br/Home/WebHome>](http://www.w3c.br/Home/WebHome) Acesso em: 14 jan.2018.



Glossário

- HTML

HyperText Markup Language ou Linguagem de Marcação de HiperTexto é o construtor de blocos mais básico da web. Ele serve para descrever e definir o conteúdo de uma página web.

Fonte: <https://developer.mozilla.org/pt-BR/docs/Web/HTML>

- HTTP

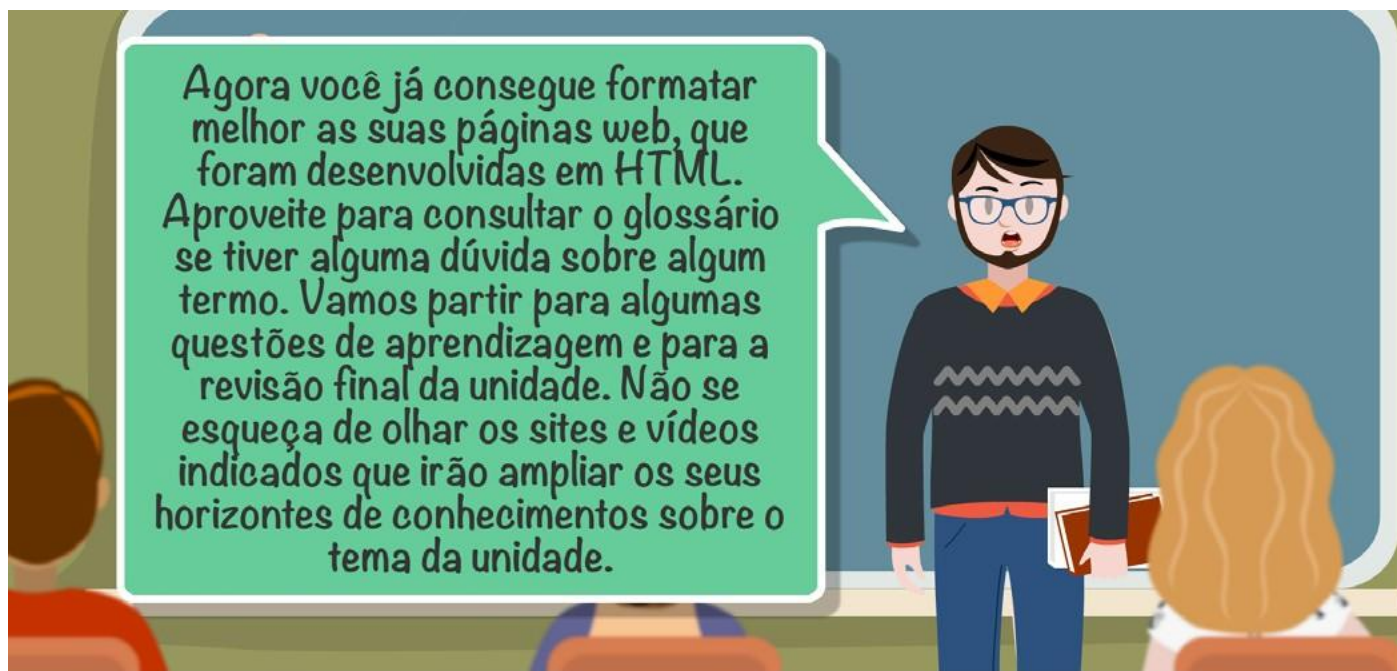
O chamado Hypertext Transfer Protocol (ou na sigla HTTP, que em tradução livre do inglês significa Protocolo de Transferência de Hipertexto) é um protocolo de comunicação (na camada de aplicação segundo o Modelo OSI) utilizado para sistemas de informação de hipermídia distribuídos e colaborativos. Seu uso para a obtenção de recursos interligados levou ao estabelecimento da World Wide Web.

Fonte: <https://www.portaleducacao.com.br/conteudo/artigos/idiomas/o-protocolo-http/20408>

Neste capítulo aprendemos os principais conceitos e as principais marcações do HTML. Agora você já consegue criar os seus próprios sites. Aproveite para consultar o glossário se tiver alguma dúvida sobre algum termo. Vamos partir para algumas questões de aprendizagem e para a revisão final da unidade. Não se esqueça de olhar os sites e vídeos indicados que irão ampliar os seus horizontes de conhecimentos sobre o tema da unidade.

E na próxima unidade professor o que vamos aprender?

Na Unidade III, vamos continuar colocando a mão na massa, porém aprenderemos como formatar a sua página, usando folhas de estilo ou CSS (como é mais conhecido). Ao final você terá uma boa base para continuar seus estudos rumo ao crescimento profissional. Espero você na próxima lição!





Curso Técnico em Informática

Fundamentos de Web Design

Unidade 3

INTRODUÇÃO AO CSS

Olá novamente!!! Na Unidade II, a gente viu como criar códigos HTML e até alguma formatação. Entretanto, essa não é a forma mais prática e produtiva uma página web. O melhor seria usar uma folha de estilos.

Por isso, nesta unidade, veremos como utilizar as folhas de estilo em cascata (Style Sheets, ou simplesmente CSS). Assim, você poderá facilmente aplicar formatação a todas as páginas de um site qualquer, o que também facilita de ajustes e eventuais manutenções, pois faz a correção em um único arquivo CSS se repete em todos os sites que usam o mesmo arquivo CSS.

Ao final desta unidade, você conhecerá os principais conceitos sobre folhas de estilo, conhecerá os principais editores, saberá usar classes e id's, aprenderá as maneiras de incluir as formatações do CSS, identificará as diversas formas de cores, saberá como alterar os padrões de fundo, saberá aplicar formatações a por fim você saberá como fazer animações simples usando o CSS.



Você sabe o que é CSS?

Passamos agora a analisar em maiores detalhes a definição de CSS (Cascading Style Sheets).

As folhas de estilo especificam como os elementos de uma página HTML serão exibidos na tela, separando o conteúdo do formato. Se utilizamos o CSS teremos os seguintes benefícios:

- Melhorar o controle da aparência de cada elemento HTML;
- Facilitar a manutenção, pois podemos fazer a alteração em um único arquivo CSS e utilizar a mesma formatação em todas as páginas de um portal;
- Permitir o desenvolvimento de páginas mais leves, facilitando o seu carregamento.



Saiba mais!

Bootstrap é o mais popular framework HTML, CSS e JS para desenvolvimento de projetos responsivo e focado para dispositivos móveis na web. Um site responsivo permite que o conteúdo se adapte ao tamanho da tela,

mudando a aparência e disposição dos elementos. Veja as configurações CSS em geral,

classes CSS para elementos HTML fundamentais e um poderoso sistema de grid.

Fonte: <http://getbootstrap.com.br/>

3.1. Editores CSS

Agora que já falamos sobre a importância do CSS, vamos aprender os códigos e colocar em prática. Vamos formatar uma página web usando o CSS.

Por onde você deve começar? Afinal, qual aplicativo você precisa usar para criar um código CSS?

A rigor, basta ter um editor de texto simples (bloco de notas) para criar um código CSS (neste caso você deve salvar como um arquivo .css) e um navegador para visualizar o resultado do referido código. Existem dezenas de ambientes de desenvolvimento. Vou apresentar para você os principais.

Nome	Breve descrição	Onde encontrar?
Adobe Dreamweaver	Famoso programa para desenvolvimento de páginas web.	http://www.adobe.com/br/products/dreamweaver.html
Notepad++	Editor de código-fonte gratuito, regido pela Licença GPL ¹ .	https://notepad-plus-plus.org/
Sublime Text	Semelhante ao Notepad++, porém com uma interface melhorada.	https://www.sublimetext.com/
Brackets	Um editor de códigos leve e limpo com ótimas funções de apoio.	brackets.io/
NetBeans	Ambiente de desenvolvimento integrado gratuito e de código aberto para desenvolvedores de software de várias linguagens.	https://netbeans.org/downloads/

1- GNU General Public License (Licença Pública Geral GNU), GNU GPL ou simplesmente GPL. Fonte: <http://www.gnu.org/>

Exemplo:

Peço que você teste todos os exemplos que mostraremos. Para isso, faça o seguinte:

1. Abra a primeira janela do aplicativo chamado “bloco de notas”;
2. Escreva o código HTML a seguir;
3. Salve o arquivo com o nome exemplo.html;
4. Abra uma segunda janela do “bloco de notas”;
5. Salve o arquivo com o nome teste.css;
6. Dê dois cliques em cima do arquivo exemplo.html
7. O navegador padrão do seu computador será aberto e você verá o resultado.

Código HTML	Código CSS
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> A cor deste texto foi definida no CSS </body> </html> </pre>	<pre> body{ color:red; } </pre>
Resultado na Tela	
	

2. Sintaxe básica e seletores

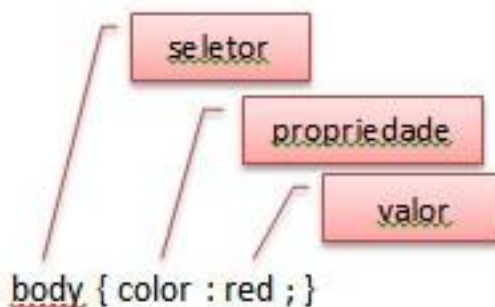
Você, uma pessoa inteligente, já entendeu o que precisa usar para trabalhar com o CSS. Agora, vamos conhecer a estrutura essencial desse tipo de código.

A sintaxe básica do CSS é composta basicamente de um seletor e um bloco de declarações, delimitado por chaves. O seletor indica o elemento do HTML que será formatado.

No bloco de declarações, temos o seguinte:

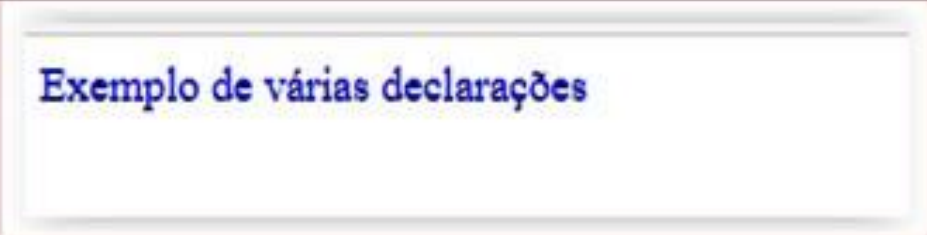
- uma ou mais propriedades, seguida de dois pontos;
- o valor de cada propriedade, seguida de ponto e vírgula.

No primeiro exemplo apresentamos o seguinte:



Em outras palavras, toda página HTML que referenciar esse código CSS apresentará em vermelho todos os textos que estiverem entre as marcações `<body>` e `</body>`.

Se você quiser fazer várias declarações para o mesmo seletor, ou seja, se quiser definir várias formatações para a mesma marcação do HTML, precisa separar cada declaração por um ponto e vírgula.

Código HTML	Código CSS
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> Exemplo de várias declarações </body> </html> </pre>	<pre> body{ color:blue; text-shadow: 0px 1px black; } </pre>
Resultado na Tela	
	

3. Classes e Id's

Você conheceu a sintaxe básica do CSS. Agora, vamos aprender como usar algumas estruturas que permitem a aplicação de determinada formatação utilizando classes e id's.



Você Sabia? (curiosidades)

“Em HTML e CSS, há a possibilidade de aplicar estilos através de ‘class’ e ‘id’ e, em JavaScript é possível indentificar algum elemento de uma página por sua classe, id ou tag. Mas qual a diferença entre ‘class’ e ‘id’?

As classes são uma forma de identificar um grupo de elementos. Através delas, pode-se atribuir formatação a VÁRIOS elementos de uma vez.

As ids são uma forma de identificar um elemento e devem ser ÚNICAS para cada elemento. É como se fossem impressões digitais de nossos dedos ou RGs. Através delas, pode-se atribuir formatação a um elemento em especial”.

Fonte: <http://tableless.github.io/iniciantes/manual/css/class-id.html>

1. Usando o id

Se você definir no CSS que os parágrafos serão na cor vermelha, todos serão vermelhos. E se você quiser que um parágrafo específico seja na cor azul?

Nesse caso, você poderia usar um id, que deve ser único, como o CPF de um brasileiro, que é único. Você define a formatação de um determinado id no CSS e utiliza o referido id com um atributo de

uma marcação do HTML. Para criar um id, basta fazer o seguinte:

- No CSS: colocar o símbolo # seguido de um nome que será usado no HTML para identificar o

Exemplo:

Código HTML	Código CSS
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> PERSONAGENS DE DRAGON BALL Z <n>Goku</n> <p id="identificador1">Vegeta</p> <p id="identificador2">Freeza</p> </body> </html> </pre>	<pre> p{ color: pink; } #identificador1{ color: blue; } #identificador2{ color: red; } </pre>
Resultado na Tela	
	

Se usar o mesmo id para mais de um marcador, seu código não vai passar pelo validador. Segundo a W3C, a maioria dos documentos da Web são escritos usando linguagens de marcação, como o HTML, que são definidas por especificações técnicas, que geralmente incluem uma gramática formal (e vocabulário) legível por máquina. O ato de verificar um documento contra essas restrições é chamado de validação, e isso é o que o Validador de Marcação faz. A validação de documentos da Web é um passo importante que pode ajudar dramaticamente a melhorar e a garantir a sua qualidade, e pode economizar muito tempo e dinheiro.

(Fonte: <https://validator.w3.org/about.html>).

2. Usando o class

Se você quiser agrupar características semelhantes, deve usar classes. Assim, você poderia aplicar a mesma formatação diferenciada em diferentes marcações. Pode parecer um pouco complicado entender, mas na prática é bem mais fácil.

Para usar uma classe, você define a formatação de uma determinada classe no CSS e a utiliza com

um atributo de uma marcação do HTML. Para criá-la, basta fazer o seguinte:

- No CSS: colocar o símbolo . seguido de um nome que será usado no HTML para identificar a classe.
- No HTML: colocar o atributo class seguido do sinal = e o nome identificador entre aspas duplas.



Exemplo:

Código HTML	Código CSS	
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> PERSONAGENS DE DRAGON BALL Z <n>Goku</n></div> <p class="minhaclasse">Vegeta</p> <p class="minhaclasse">Freeza</p> </body> </html> </pre>	<pre> p{ color:red; } .minhaclasse{ color:blue; } </pre>	
Resultado na Tela		
		

4. Maneiras de incluir estilos

Imagine a seguinte situação: você foi contratado(a) para desenvolver um grande portal. Parabéns!!! Então você terá que criar e formatar 45 páginas HTML. Imagine ter que formatar cada página individualmente. E o problema se agrava quando estamos na fase de homologação, quando temos que testar e corrigir todas as páginas. Um trabalho maior do que a batalha entre Goku e Freeza, que durou 20 episódios (risos).

Nesse caso, seria melhor você entender as diferentes formas de inserir as formatações do CSS: linking, embedding e inline.

Você já entendeu como usar classes e id's. Parabéns!!! Você é uma pessoa inteligente. Agora vamos aprender que podemos inserir as formatações do CSS de diferentes formas.

Você pode formatar um documento HTML de acordo com as informações de uma folha de estilo (CSS), que podem ser inseridas de três formas diferentes, a saber:

- Linking (externa ou lincada): carrega um arquivo CSS, que pode ser usado em várias páginas, utilizando a marcação link;
- Embedding (interna ou incorporada): define um código CSS na própria página HTML, sem afetar outras páginas, utilizando a marcação style.
- Inline (na linha): define a formatação do CSS apenas em uma marcação específica, utilizando

a propriedade style.



Você Sabia? (curiosidades)

Imagine a seguinte situação: você criou um site com dezenas de páginas HTML. Em cada página você colocou um link para um arquivo externo de

CSS. Entretanto, em uma única página você quer fazer uma alteração específica, ou seja, não quer mudar tudo, apenas um detalhe. Nesse caso, qual regra será seguida? A regra do arquivo externo ou a regra da página específica?

Em ordem de hierarquia, a formatação inline se sobrepõe à formatação incorporada, que se sobrepõe à formatação linkada.

Fonte: <http://tableless.github.io/iniciantes/manual/css/class-id.html>



CSS Externo	CSS Incorporado	CSS inline
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> <p>CSS externo</p> </body> </html> </pre>	<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> <style> p{color:blue;} </style> </head> <body> <p>CSS incorporado</p> </body> </html> </pre>	<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> <style> p{color:blue;} </style> </head> <body> <p style="color:green"> CSS inline </p> </body> </html> </pre>
Resultado na Tela		
		

3.5. Uso de cores

Você já sabe como incluir o CSS. Vamos agora identificar como trabalhar com cores no CSS. Você vai notar que existem várias formas de especificar uma mesma cor.

Como você deve ter notado, podemos formatar as cores de um determinado elemento HTML, usando a propriedade color e valor da cor. Atualmente os navegadores suportam 140 cores diferentes. No CSS há diferentes formas de identificar uma cor:

- Pelo nome da cor: existem 140 diferentes nomes pré-definidos de cores. Você pode encontrar todos os nomes padrão de cores no seguinte endereço: <https://www.w3.org/TR/2011/REC-css3-color-20110607/>.

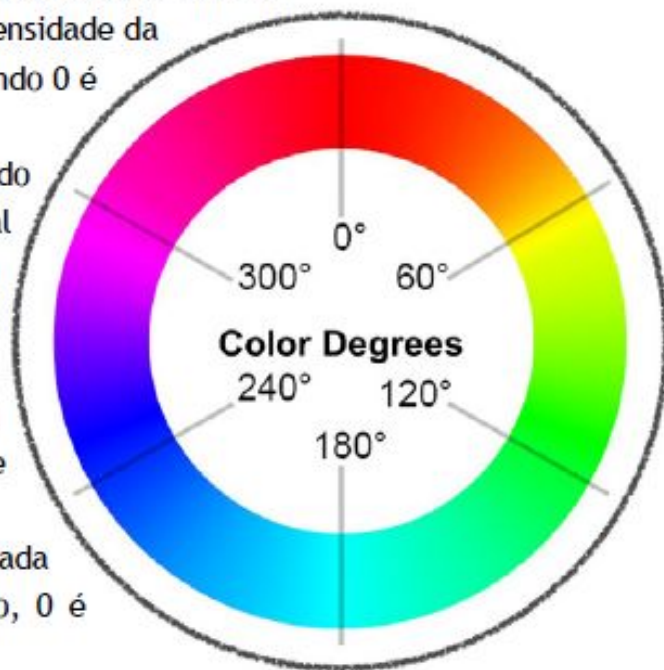
- Pelos valores em RGB: todas as cores podem ser representadas pela combinação das cores vermelho (Red em inglês), verde (green em inglês) e azul (Blue em inglês). Para cada uma dessas cores, você pode atribuir valores de 0 a 255.

- Valores em RGBA: também representa as cores usando as cores Red, Green, Blue e acrescentamos o Alfa, que indica a intensidade da transparência. O valor Alfa pode variar de 0 a 1, quando 0 é uma cor opaca e 1 seria a total transparência.

- Valores em Hexadecimal: os valores de 0 a 255 do RGB podem ser convertidos para o padrão hexadecimal. Assim, você coloca o símbolo # e os valores em hexadecimal representando os valores do padrão RGB.

- Valores em HSL: podemos definir uma cor em termos da tonalidade (Hue), saturação (Saturation) e leveza (Lightness).

o Hue é uma roda de cores, em que cada ângulo indica uma cor pura diferente. Por exemplo, 0 é vermelho, 120 é verde e 240 é azul.



- o Saturação é um valor percentual que representa a pureza ou intensidade de uma cor, variando de 0% a 100%.
- o A leveza é um valor percentual que representa a quantidade de luz percebida, variando de 0% a 100%.
- Valores em HSLA: também representa as cores usando o padrão HSL e acrescentamos o Alfa, que indica a intensidade da transparência. O valor Alfa pode variar de 0 a 1, quando 0 é uma cor opaca e 1 seria a total transparência.

Código CSS

```
#testeNome{ background-color:red; }
#testeRGB{ background-color:rgb(255,0,0); }
#testeRGBA{ background-color:rgba(255,0,0,0.5); }
#testeHex{ background-color:#ff0000; }
#testeHSL{ background-color:hsl(0,100%,50%); }
#testeHSLA{ background-color:hsla(0,100%,50%,0.5); }
```

Código HTML

```
<!DOCTYPE html>
<html>
  <title>Exemplo CSS</title>
  <head>
    <meta charset="UTF-8">
    <link href="teste.css"
      type="text/css"
      rel="stylesheet">
  </head>
```



```
<body>  
  <button id="testeNome">Cor Vermelha</button><br>  
  <button id="testeRGB">Cor Vermelha</button><br>  
  <button id="testeRGBA">Cor Vermelha</button><br>  
  <button id="testeHex">Cor Vermelha</button><br>  
  <button id="testeHSL">Cor Vermelha</button><br>  
  <button id="testeHSLA">Cor Vermelha</button><br>  
</body>  
</html>
```

Resultado na Tela





Saiba mais!

Eventualmente você vai querer converter uma determinada cor para padrões diferentes. O cálculo manual dessa conversão pode ser mais complicado. Entretanto, existem diversos sites que fazem a conversão online, um deles é o webcalc.

Disponível em: < http://webcalc.com.br/Utilitarios/form/rgb_hex >.

6. Alterando os padrões de fundo

Conforme vimos no exemplo acima, podemos definir diferentes padrões de fundo de um elemento do HTML, que pode ser uma cor específica ou uma imagem. Usando o CSS, você consegue formatar muito melhor os padrões de fundo, tanto cor quanto imagem de fundo.

1. Usando uma cor de fundo

Para definir a cor de fundo de um elemento do HTML, podemos utilizar a propriedade `background-color` com uma das especificações de cores vistas no item anterior.

Sintaxe básica:

```
seletor {background-color: cor;}
```

2. Usando uma imagem de fundo

E, se ao invés de uma cor, você quisesse colocar uma imagem de fundo?

Nesse caso, você precisaria usar estas propriedades:

- background-image

Com esta propriedade, você pode especificar a imagem que será usada como fundo do elemento do HTML. Por definição, a imagem será repetida ao longo de todo o elemento.

Exemplo:

Código HTML	Código CSS
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> </body> </html> </pre>	<pre> body{ background-image: url("imagens/tbbt.jpg"); } </pre>
Resultado na Tela	
	

Observe que você precisa indicar onde a imagem se encontra. Se estiver em uma pasta diferente de onde você salvou o código HTML, deverá indicar o caminho até a imagem.

Você também pode indicar o endereço do site onde a imagem pode ser localizada. Por

exemplo: `body{`

```
background-image: url("https://upload.wikimedia.org/wikipedia/lt/thumb/1/1b/BigBangTheoryTitleCard.png/250px-BigBangTheoryTitleCard.png");
```

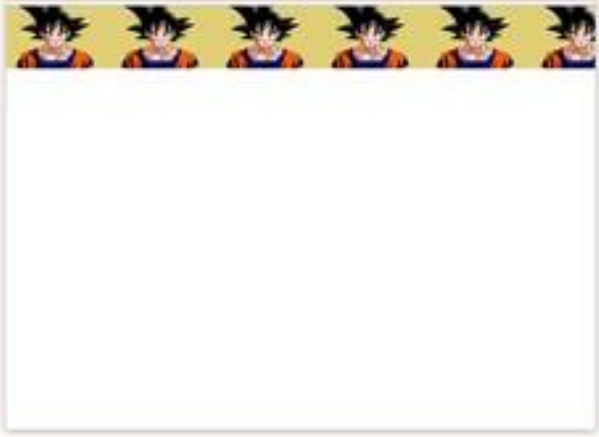
```
}
```

- **background-repeat**

E se você quiser repetir a imagem apenas na vertical ou na horizontal?

Neste caso, você deve utilizar a propriedade `background-repeat`, junto com a propriedade `background-image`. Os principais valores permitidos para essa nova propriedade são:

- o `repeat-x`: a imagem de fundo é repetida apenas horizontalmente;
- o `repeat-y`: a imagem de fundo é repetida apenas verticalmente;
- o `no-repeat`: a imagem de fundo não será repetida;
- o `space`: a imagem de fundo é repetida o máximo possível sem recortar.

Código CSS - exemplo 1	Código CSS - exemplo 2
<pre>body{ background-image: url("imagens/goku_menor.jpg"); background-repeat: repeat-x; }</pre>	<pre>body{ background-image: url("imagens/goku_menor.jpg"); background-repeat: repeat-y; }</pre>
Resultados na Tela	
	

- **background-attachment**

Usando esta propriedade, a imagem de fundo não rola com a página, fica fixa.

Código HTML	Código CSS
<pre><!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> Todos os dias quando acordo
 Não tenho mais
 O tempo que passou
 Mas tenho muito tempo
 Temos todo o tempo do mundo<p> Todos os dias
 Antes de dormir
 Lembro e esqueço
 Como foi o dia
 Sempre em frente
</pre>	<pre>body{ background-image: url("legiao.jpg"); background-repeat: no-repeat; background-attachment: fixed; }</pre>

Não temos tempo a perder<p>
Nosso suor sagrado

É bem mais belo

Que esse sangue amargo

E tão sério

E selvagem! Selvagem!

Selvagem!<p>
Veja o sol

Dessa manhã tão cinza

A tempestade que chega

É da cor dos teus olhos

Castanhos<p>
Então me abraça forte

E diz mais uma vez

Que já estamos

Distantes de tudo

Temos nosso próprio tempo

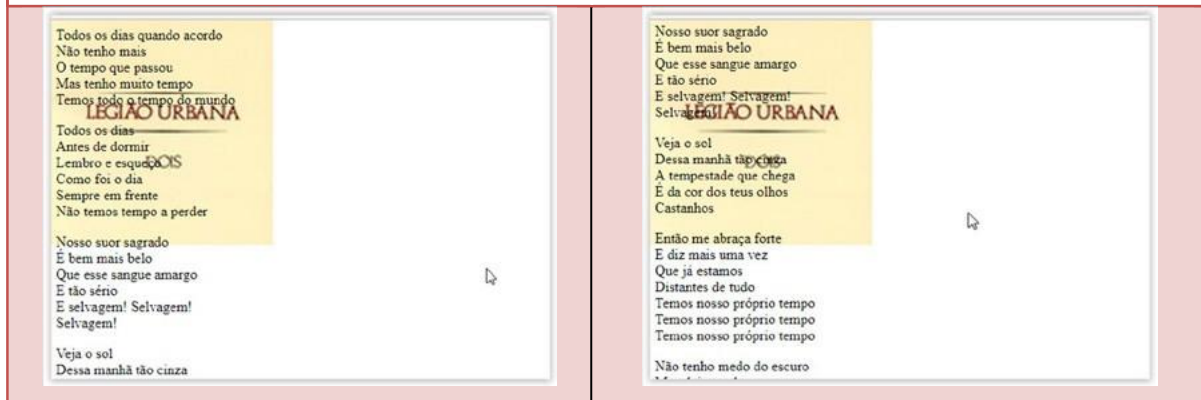
Temos nosso próprio tempo

Temos nosso próprio tempo<p>

</body>

</html>

Resultados na Tela (mostra que a imagem fica fixa, mesmo rolando o texto)



- **background-position**

Por padrão, uma imagem em segundo plano é colocada no canto superior esquerdo da tela. E se você não quiser a imagem nesse local?

A propriedade background-position define outra posição inicial de uma imagem de fundo, utilizando os valores: left top, left center, left bottom, right top, right center, right bottom, center top, center center e center bottom.



Código HTML	Código CSS
<pre><!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> Veja o sol
 Dessa manhã tão cinza
 A tempestade que chega
 É da cor dos teus olhos
 Castanhos<p> Então me abraça forte
 E diz mais uma vez
 Que já estamos
 Distantes de tudo
 Temos nosso próprio tempo
 Temos nosso próprio tempo
 Temos nosso próprio tempo<p> </body> </html></pre>	<pre>body{ background-image: url("imagens/legiao.jpg"); background-repeat: no-repeat; background-attachment: fixed; background-position:center top; } html{ background-color:gray; color:white; }</pre>
Resultados na Tela	
	



Saiba mais!

“Em se tratando de planos de fundo, as CSS nos fornecem a propriedade `background`, recurso que traz diferentes opções para customização desse espaço nos elementos declarados em nossas páginas. Com `background`

podemos definir a cor de fundo, imagem, o posicionamento dela, se deve se repetir, entre outras opções, agregando um diferencial ainda maior para o visual das nossas aplicações”. Saiba mais neste vídeo da DevMedia.

Fonte: <https://www.devmedia.com.br/css-background/3831>



7. Textos e fontes

No item anterior, você compreendeu como formatar padrões de fundo. Agora vamos aprender a formatar textos e fontes usando as propriedades do CSS.

As propriedades que formatam o texto e as fontes são as seguintes:

- **color:** define a cor de um texto, utilizando os valores já vistos anteriormente;
- **text-align:** define o alinhamento do texto na horizontal, utilizando os seguintes valores:
 - o **left:** alinhado à esquerda;
 - o **right:** alinhado à direita;
 - o **center:** alinhamento centralizado;
 - o **justify:** alinhamento justificado.
- **text-decoration:** define o uso de linhas de decoração em um texto, que pode ser:
 - o **overline:** acima do texto;
 - o **underline:** abaixo do texto;
 - o **line-through:** em cima do texto.
- **text-transform:** define o uso de letras em caixa alta, que pode conter os seguintes valores:
 - o **uppercase:** todas as letras em caixa alta;
 - o **lowercase:** todas as letras em caixa baixa;
 - o **capitalize:** apenas a primeira letra em caixa alta.
- **letter-spacing:** define o espaçamento (em pixels) entre os caracteres;
- **line-height:** define o espaçamento (em pixels) entre as linhas;
- **word-spacing:** define o espaçamento (em pixels) entre as palavras;
- **text-shadow:** insere uma sombra ao texto. Você deve especificar:
 - o a posição da sombra no sentido horizontal (em pixels);
 - o a posição da sombra no sentido vertical (em pixels); e
 - o a cor da sombra.
- **font-family:** define a família de fontes a ser utilizada. Se o navegador não suportar a primeira,

ele tenta a próxima, e assim por diante.

- **font-style:** define se o texto será mostrado em itálico ou não, usando o valor `italic`;
- **font-size:** define o tamanho da fonte, que pode ser especificado em termos:
 - o Absolutos: define a fonte com um tamanho específico (em pixels ou cm);
 - o Relativos: define a fonte com um tamanho relativo. Usando:
 - a unidade de medida em “em”, em que 1em é o tamanho padrão;
 - `xx-small`: define um tamanho muito muito menor que o padrão;
 - `x-small`: define um tamanho muito menor que o padrão;
 - `small`: define um tamanho menor que o padrão;
 - `large`: define um tamanho maior que o padrão;
 - `x-large`: define um tamanho muito maior que o padrão;
 - `xx-large`: define um tamanho muito muito maior que o padrão;
 - `smaller`: define um tamanho menor que o padrão;
 - `larger`: define um tamanho maior que o padrão
 - `%`: define um valor percentual em relação ao tamanho padrão.
- **font-weight:** define o peso da fonte, ou seja, a intensidade do negrito. Pode utilizar um destes valores:
 - o `bold`: define o negrito padrão;
 - o `bolder`: define um negrito mais intenso;
 - o `lighter`: define um destaque usando um negrito menos intenso que o padrão;
 - o valores de 100 a 900.

Código HTML	Código CSS
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> <p class="formato1"> linha vermelha centralizada </p> linha antes <p class="formato2"> caixa alta e letras espaçadas e linhas distantes </p> linha depois <p class="formato3"> espaçamento entre as palavras e sombra azul </p> <p class="formato4"> fonte courier itálico grande e em negrito </p> </body> </html> </pre>	<pre> p.formato1{ color:red; text-align:center; text-decoration:underline; } p.formato2{ text-transform:uppercase; letter-spacing:10px; line-height:90px; } p.formato3{ word-spacing: 10px; text-shadow:1px 1px blue; } p.formato4{ font-family:courier,"Times New Roman"; font-style:italic; font-size:xx-large; font-weight:900; } </pre>

Resultados na Tela

Antes

CAIXA ALTA E LETRAS ESPAÇADAS E LINHAS DISTANTES

Depois

espaçamento entre as palavras e fontes azul

fonte courier itálico grande e em negrito



Saiba mais!

A MDN, uma rede de desenvolvedores da Mozilla, é uma plataforma de aprendizagem sobre as principais tecnologias da Web, que disponibiliza um tutorial CSS, do qual destacamos a página referente aos estilos de texto. Disponível em: < https://developer.mozilla.org/pt-PT/docs/Web/CSS/Como_come%C3%A7ar/Estilos_de_texto >.

3.8. Animação com CSS



Bora Refletir!

Já imaginou fazer uma animação bem legal, fazendo a transição de estilo CSS para outro? Por exemplo, fazendo a transição suave de cores de fundo de um elemento HTML qualquer. É muito simples. Vamos aprender?

Para a fazer uma animação, o mínimo que você precisa fazer é o seguinte:

- Definir regras dentro de um @keyframe: essas regras especificam os percentuais de mudança

de estilo. Para que uma animação funcione, você deve vincular a animação a um elemento.

- Definir os critérios da animação usando:
 - o animation-name: especifica um nome para a animação @keyframes;
 - o animation-duration: define quantos segundos ou milissegundos uma animação leva para completar um ciclo;
 - o animation-delay: especifica um atraso em segundos ou milissegundos para o início de uma animação;
 - o animation-direction: define se uma animação deve tocar no sentido inverso ou em ciclos alternados. Use alternate ou reverse;
 - o animation-iteration-count: especifica o número de vezes que uma animação deve ser reproduzida. Use um número específico de vezes ou infinite.

1. Transformações

Junto com as animações você também pode fazer transformações 2D, usando a propriedade `transform` e os seguintes valores:

- `translate()`: que move um elemento HTML da sua posição atual (de acordo com os parâmetros dados para o eixo X e o eixo Y). Exemplo: `translate(10px,10px)`;
- `rotate()`: que gira um elemento HTML no sentido horário ou anti-horário de acordo com um determinado grau. Exemplo: `rotate(10deg)`;
- `scale()`: que aumenta ou diminui a escala de um elemento HTML. Exemplo: `scale(1.5,1.5)`.



Código HTML	Código CSS
<pre> <!DOCTYPE html> <html> <title>Exemplo CSS</title> <head> <meta charset="UTF-8"> <link href="teste.css" type="text/css" rel="stylesheet"> </head> <body> <button></button> </body> </html> </pre>	<pre> @keyframes aula { 0% {transform: translate(0px,0px);} 20% {transform: translate(200px,0px);} 40% {transform: translate(400px,0px);} 60% {transform: translate(600px,0px);} 80% {transform: translate(800px,0px);} 100% {transform: translate(1000px,0px);} } button { background-color: red; animation-name: aula; animation-duration: 2s; animation-iteration-count: infinite; animation-direction: alternate; width: 100px; height: 100px; } </pre>
Resultados na Tela	
	



Questões de autoaprendizagem

Vamos pensar e praticar...

Vamos voltar àquele exercício de HTML, no qual você criou uma tabela com seu desenho, jogos e Youtuber favoritos. Tenho certeza que você conseguiu fazer. Afinal, você é uma pessoa muito inteligente!!!

Agora, vamos formatar aquela página usando o CSS. Você deve alterar, no mínimo:

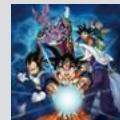
- cores de fundo;
- tamanho e negrito das fontes;
- expandir a largura da tabela para 100%;
- inserir animações diferentes para as 3 imagens.

Novamente, seja criativo(a), faça algo legal e mostre para seus/ suas amigos(as). Tenho certeza de que você consegue. Atenção: as imagens são apenas ilustrativas.

Coisas que eu gosto



Coisas que eu gosto



Solução...

Para resolver essa questão, você deveria pensar no padrão de cores que iria usar, alterar as fontes e imaginar as animações de translação, mudança de escala e rotação. Por fim, deveria escrever um código semelhante a este:

Arquivo HTML

```
<html>
  <head>
    <title>Exemplo</title>
    <meta charset="UTF-8">
    <link rel="stylesheet"
      href="exemplo.css"
      type="text/css">
  </head>
  <body>
    <table border>
      <tr>
        <th colspan="2">Coisas que eu gosto</th>
      </tr>
      <tr>
        <td>Meu desenho favorito</td>
        <td>
          
          </td>
        </tr>
    </table>
  </body>
</html>
```

```

<tr>
  <td>Meu game preferido</td>
  <td>
    
</tr>
<tr>
  <td>Meu youtuber preferido</td>
  <td>
    
</tr>
<tr>
  </td>
</tr>
</table>
</body>
</html>

```

Arquivo CSS

```
table{
  width:100%;
}
th{
  background-color:rgb(12,97,18)
  ; color:white;

  text-align:center;
  text-transform:uppercase
  ; font-size:xx-large;
}
td{
  background-color:rgb(187,247,191)
  ; text-align:center;
  text-shadow: 1px 1px
  green; font-weight: 600;

  font-size:30px;
}
@keyframes animaYoutuber {
  8% {transform: rotate(30deg);}
  17% {transform: rotate(60deg);}
  25% {transform: rotate(90deg);}
  33% {transform: rotate(60deg);}
  42% {transform: rotate(30deg);}
  50% {transform: rotate(0deg);}
```



```
58% {transform: rotate(-30deg);}
66% {transform: rotate(-60deg);}
75% {transform: rotate(-90deg);}
83% {transform: rotate(-60deg);}
91% {transform: rotate(-30deg);}
100% {transform: rotate(0deg);}
}
@keyframes animaJogo {
  13% {transform: scale(0.9,0.9);}
  25% {transform: scale(0.7,0.7);}
  38% {transform: scale(0.5,0.5);}
  50% {transform: scale(0.3,0.3);}
  63% {transform: scale(0.5,0.5);}
  75% {transform: scale(0.7,0.7);}
  88% {transform: scale(0.9,0.9);}
  100% {transform: scale(1.0,1.0);}
}
@keyframes animaDesenho {
  13% {transform: translate(30px,0px);}
  25% {transform: translate(60px,0px);}
  38% {transform: translate(30px,0px);}
  50% {transform: translate(0px,0px);}
  63% {transform: translate(-30px,0px);}
  75% {transform: translate(-60px,0px);}
  88% {transform:
    translate(-3
    0px,0px);}
  100% {transform: translate(0px,0px);}
}
```

```
#giraYoutuber{
  animation-name:
  animaYoutuber;
  animation-duration: 3s;
  animation-iteration-count:infinite;
}

#encolheJogo{
  animation-name:
  animaJogo;
  animation-duration: 3s;
  animation-iteration-count:infinite;
}

#deslizaDesenho{
  animation-name:
  animaDesenho;
  animation-duration: 3s;
  animation-iteration-count:infinite;
```



Síntese final do módulo (Bora rever!)

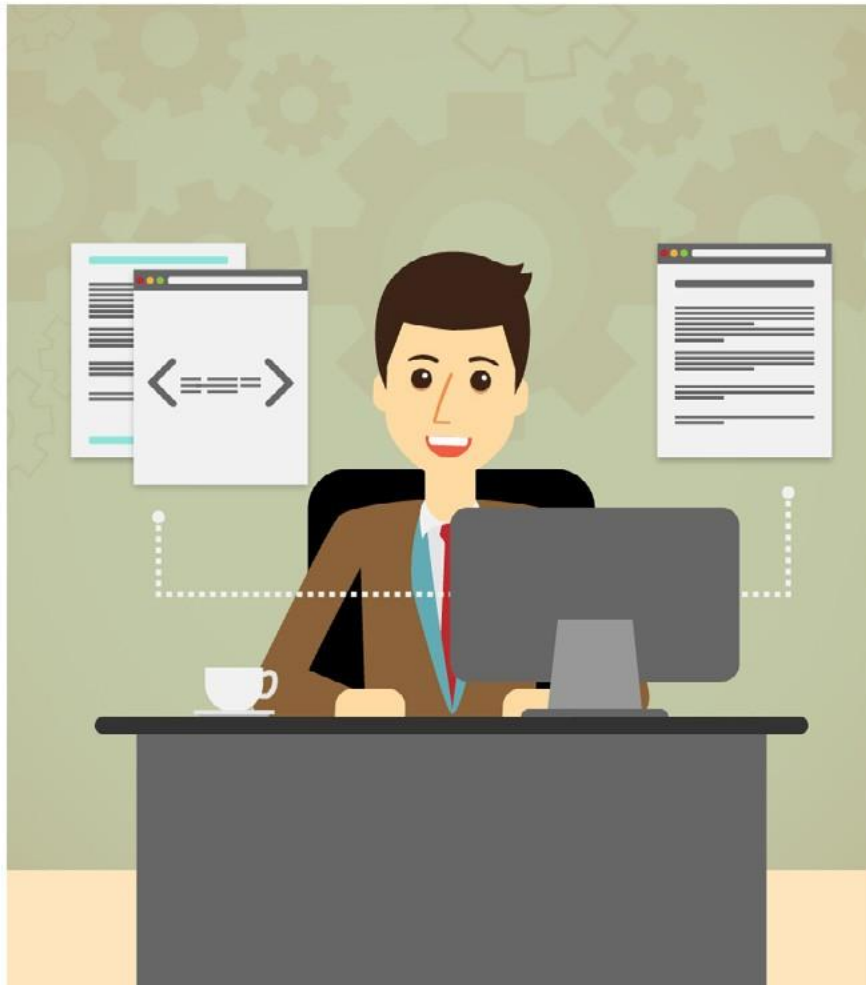
Bom, chegamos ao final desta unidade! Espero que você tenha gostado. Você, uma pessoa muito inteligente e divertida, conseguiu compreender os principais conceitos sobre folhas de estilo, conheceu os principais editores,

aprendeu a usar classes e id's, descobriu diversas maneiras de incluir as formatações do CSS, aprendeu as diversas formas de uso de cores, conheceu as técnicas para alterar os padrões de fundo, entendeu como aplicar formatações a textos e fontes, e aprendeu a fazer animações simples usando o CSS.



Sites indicados

- CSS Tutorial. Disponível em:
<<https://www.w3schools.com/css/default.asp>>. Acesso: 15jan. 2018.
- CSS. Disponível em: < <https://developer.mozilla.org/pt-BR/docs/Web/CSS>>. Acesso: 15jan. 2018.





Glossário

- Background

É um substantivo masculino que significa fundo ou segundo plano.

Fonte: <https://www.dicio.com.br/background/>

- CSS

CSS (Cascading Style Sheets ou em português: folhas de estilos em cascata) é uma linguagem de estilo usada para descrever a apresentação de um documento escrito em HTML ou em XML (incluindo várias linguagens em XML como SVG ou XHTML). O CSS descreve como elementos são mostrados na tela, no papel, no discurso ou em outras mídias.

Fonte: <https://developer.mozilla.org/pt-BR/docs/Web/CSS>

- Padding

É uma propriedades CSS usada para gerar espaço em torno do conteúdo de um elemento, dentro de qualquer limite definido.

Fonte: https://www.w3schools.com/css/css_padding.asp

- Template É um modelo a ser seguido, com uma estrutura predefinida que facilita o desenvolvimento

e a criação do conteúdo a partir de algo construído a priori. Ao instalar o Joomla! em sua versão mais básica, alguns modelos já são trazidos e podem ser usados e modificados livremente, mas dependendo do que se deseja construir ou modificar, existem outros modelos que melhor se adaptam à nossa necessidade.

Fonte:

<https://www.portaleducacao.com.br/conteudo/artigos/educacao/o-que-e-template/39828>

Assim terminamos nossa Unidade 3 – CSS. Espero que tenham compreendido o assunto. Nesta unidade aprendemos os principais conceitos e as principais propriedades do CSS. Agora você já consegue formatar melhor as suas páginas web, que foram desenvolvidas em HTML. Aproveite para consultar o glossário se tiver alguma dúvida de algum termo. Vamos partir para algumas questões de aprendizagem e para a revisão final da unidade. Não esqueça de olhar os sites e vídeos indicados que irão ampliar os seus horizontes de conhecimentos sobre o tema da unidade.

E na próxima unidade professor o que vamos aprender?

Na Unidade IV vamos continuar colocando a mão na massa, porém aprenderemos como desenvolver códigos em JavaScript, fechando os conhecimentos básicos na área de web design. Ao final você terá uma boa base para continuar seus estudos rumo ao crescimento profissional. Espero você na próxima lição!





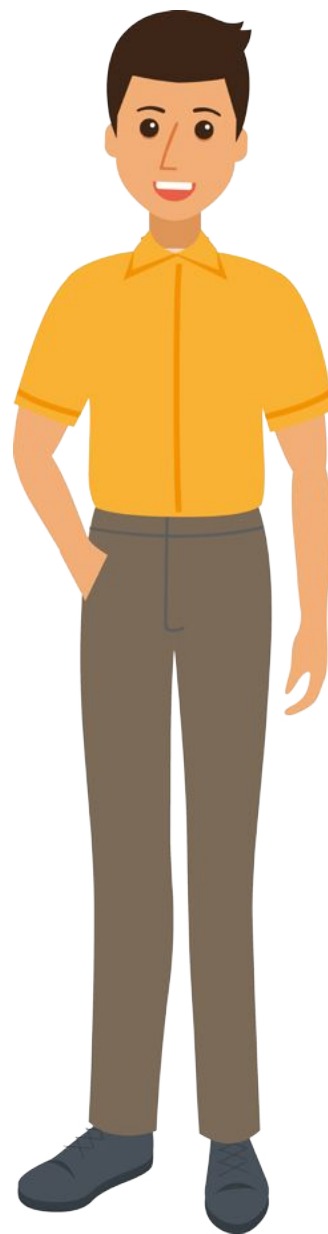
Unidade 4

JavaScript

Olá, que bom encontrar você!!! Vamos avançar mais um pouco nos fundamentos de web design? Na Unidade 3, a gente viu como formatar uma página HTML usando as propriedades do CSS. Agora, vamos aprender a utilizar o JavaScript, uma linguagem extremamente versátil e essencial para quem trabalha com desenvolvimento web.

Por exemplo, você já preencheu um formulário na web e quando deixou um campo em branco, recebeu um aviso para preencher? Então, isso é feito com o JavaScript, que é uma linguagem de programação codificada no interior de página HTML ou em um arquivo separado.

Ao final desta unidade, você conhecerá os principais conceitos sobre o JavaScript, conhecerá os principais editores, aprenderá as diversas maneiras de incluir a linguagem JavaScript em sua página web, conhecerá várias maneiras de mostrar uma informação na tela usando essa linguagem, saberá quais são os elementos básicos de programação, aprenderá a criar funções e eventos com o JavaScript.





Você Sabia? (curiosidades)

Muita gente acha que JavaScript é uma espécie de versão da Linguagem Java, mas não é. A linguagem de programação JavaScript, desenvolvida

pela Netscape, não faz parte da plataforma Java. O JavaScript não cria applets ou aplicações independentes. Na sua forma mais comum, o JavaScript fica embutido nos documentos HTML e pode fornecer níveis de interatividade para páginas Web que não são acessíveis com um HTML simples.

Diferenças-chave entre o Java e o JavaScript:

- Java é uma linguagem de programação Orientada a Objeto, ao passo que JavaScript é uma linguagem de scripts Orientada a Objeto.
- Java cria aplicações executadas em uma máquina virtual ou em um browser, ao passo que o código JavaScript é executado apenas em um browser.
- O código Java precisa ser compilado, ao passo que os códigos JavaScript estão totalmente em texto.
- Eles requerem plug-ins diferentes.

(Fonte: https://www.java.com/pt_BR/download/faq/java_javascript.xml)

4.1. Editores

Desenvolvedores profissionais utilizam Ambientes de Desenvolvimento Integrado (em inglês, Integrated Development Environment - IDE), que devem oferecer um conjunto de ferramentas que facilitam o trabalho do desenvolvedor, incluindo um bom editor de texto, sintaxes destacadas, funcionalidades de auto completar e teclas de atalho.

Sabemos que existem diversas IDE's, então por onde você deve começar? Afinal, qual aplicativo você

precisa usar para criar um código em JavaScript? Vejamos os principais ambientes logo a seguir.

A rigor, basta ter um editor de texto simples (bloco de notas) para criar um código CSS (neste caso, você deve salvar como um arquivo .css) e um navegador para visualizar o resultado do referido código. Existem dezenas de ambiente de desenvolvimento. Vou apresentar para você os principais.

Você precisa de algum aplicativo para criar um código em JavaScript?

Semelhante ao que dissemos na unidade sobre HTML, para criar um código em JavaScript basta ter um editor de texto simples (bloco de notas), salvar como um arquivo .js e um navegador para visualizar o resultado do referido código. Entretanto, existem dezenas de ambientes de desenvolvimento, os mesmos do HTML, vamos lembrar.

Nome	Breve descrição	Onde encontrar?
Adobe Dreamweaver	Este é provavelmente o mais famoso programa para desenvolvimento de páginas web.	http://www.adobe.com/br/products/dreamweaver.html
Notepad++	É um editor de código-fonte gratuito, regido pela Licença GPL ¹ .	https://notepad-plus-plus.org/
Sublime Text	Semelhante ao Notepad++, porém com uma interface melhorada.	https://www.sublimetext.com/
Brackets	Um editor de códigos leve e limpo com ótimas funções de apoio.	brackets.io/
NetBeans	É um ambiente de desenvolvimento integrado gratuito e de código aberto para desenvolvedores de software de várias linguagens.	https://netbeans.org/downloads/

1- GNU General Public License (Licença Pública Geral GNU), GNU GPL ou simplesmente GPL.

Fonte: <http://www.gnu.org/>

Exemplo:

Peço que você teste todos os exemplos que mostraremos. Para isso, faça o seguinte:

1. Abra o aplicativo chamado “bloco de notas”;
2. Escreva o código a seguir;
3. Salve o arquivo com o nome exemplo.html;
4. Dê dois cliques em cima do arquivo exemplo.html;
5. O navegador padrão do seu computador será aberto e você verá o resultado.



Você Sabia? (curiosidades)

O código em JavaScript pode ser escrito tanto no interior de um código HTML, quanto em um arquivo externo. Veremos como fazer isso daqui a pouco!

O código em JavaScript pode ser colocado dentro do código HTML, entre as seções <body> ou <head>, utilizando a marcação <script>, usando o parâmetro type, com o valor text/javascript.

Código HTML com JavaScript

```
<html>
  <body>
    <script type="text/javascript">
      alert("primeiro JavaScript");
    </script>
  </body>
</html>
```

Resultado no Navegador



4.2. O JavaScript em um arquivo externo



Bora Refletir!

Lembra que a gente usou o elemento `<link>` do HTML para referenciar um arquivo externo do CSS. Então, com o JavaScript é bem semelhante.

Assim a gente pode utilizar o mesmo código em JavaScript em várias páginas HTML, de maneira rápida, fácil e eficiente, facilitando também a manutenção do referido código.

Escreva seu código JavaScript, salve em um arquivo com a extensão `.js` com todas as instruções do

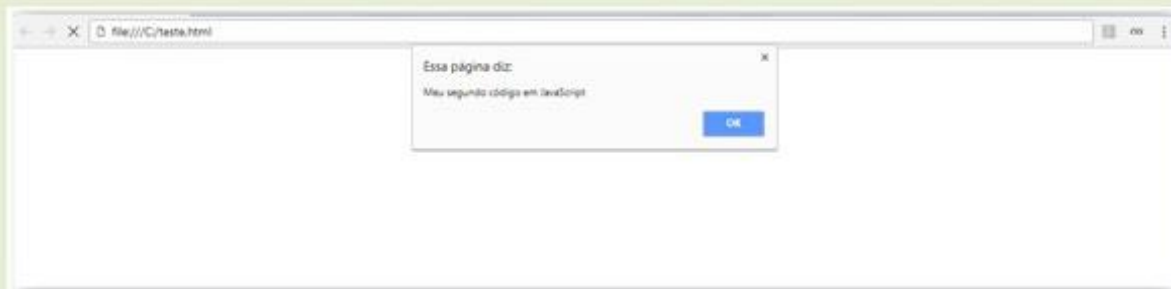
JavaScript. No código HTML, chame o referido arquivo `.js` da seguinte forma:

- Use a marcação `script`, a propriedade `type` e o valor `text/javascript`.
- Na mesma marcação `script`, use também a propriedade `src`, e como valor da mesma, use o nome do arquivo junto com a extensão.

Exemplo:

Código HTML	Código JavaScript Externo
<pre><!DOCTYPE html> <html> <title>Título da Página</title> <head> <script charset="ISO-8859-1" type="text/javascript" src="exemplo.js"> </script> </head> <body> </body> </html></pre>	<pre>alert("Meu segundo código em JavaScript");</pre>

Resultado no Navegador



Você Sabia? (curiosidades)

O atributo do charset com o valor “ISO-8859-1” é usado para indicar que formatação utilizará a codificação de caracteres do alfabeto latino.

4.3. Como mostrar informações na tela?

No JavaScript, assim como em outras linguagens, temos uma coisa chamada função, que é basicamente um bloco de código que realiza uma tarefa específica. Esse bloco facilita a manutenção do código, mas só é executado quando é chamado.

Em outro tópico, falaremos como criar as suas próprias funções. Neste momento utilizaremos funções pré-existentes, mais especificamente funções que mostram informações na tela do navegador.

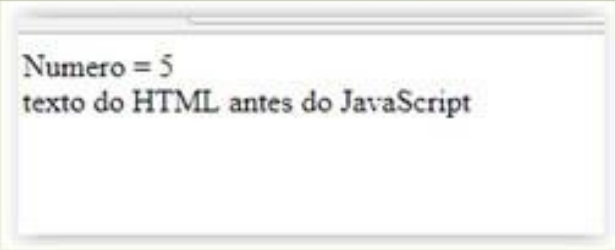
4.3.1. Usando a função alert()

Você já deve ter observado que utilizamos essa função no exemplo anterior para abrir uma janela e mostrar uma mensagem de alerta. Adicionalmente, você pode concatenar um texto com um valor numérico usando o símbolo +.

Código HTML	Código JavaScript Externo
<pre><!DOCTYPE html> <html> <title>Título da Página</title> <head> <script charset="ISO-8859-1" type="text/javascript" src="exemplo.js"> </script> </head> <body> </body> </html></pre>	<pre>alert("Numero = "+5);</pre>
Resultado no Navegador	
	

4.3.2. Usando a document.write ()

Funciona de forma semelhante ao alert, mas em vez de apresentar o texto em uma janela, apresenta na própria página web.

Código HTML	Código JavaScript Externo
<pre><!DOCTYPE html> <html> <title>Título da Página</title> <head> <script charset="ISO-8859-1" type="text/javascript" src="exemplo.js"> </script> </head> <body> texto do HTML antes do JavaScript
 </body> </html></pre>	<pre>document.write("Numero = "+5);</pre>
Resultado no Navegador	
	

4.3.3. Usando a função getElementById() com o innerHTML

Você lembra que a gente estudou o uso do id unidade III, quando falamos sobre o CSS? Foi dito que id é uma forma de identificar um elemento do HTML. Dissemos também que o id deve ser único para cada elemento.

E se você quisesse mostrar alguma informação em um elemento específico do HTML usando esse id? Nesse caso, você poderia utilizar o método getElementById() do objeto document, junto com a propriedade innerHTML. Essa propriedade possibilita a troca do conteúdo de um determinado elemento, sem a necessidade de recarregar página web.

Vamos apresentar um exemplo a seguir e se você quiser ver mais detalhes acesse estes sites:

document.getElementById(). Disponível em: <
<https://developer.mozilla.org/pt-BR/docs/Web/API/Document/getElementById> >. Acesso:
19jan.2018.

Element.innerHTML. Disponível em:
<<https://developer.mozilla.org/pt-BR/docs/Web/API/Element/innerHTML> >. Acesso:
19jan.2018.

A sintaxe básica é a seguinte:

`document.getElementById(id).innerHTML = valor`

Onde:

- `getElementById()`: é a função que retorna, por meio do id, a referência de um determinado

elemento;

- `id`: é o identificador único que você utilizou em um elemento do HTML;
- `innerHTML`: é a propriedade que substitui o elemento do HTML;
- `valor`: é o conteúdo que será utilizado para substituir o conteúdo do elemento identificado

pelo id.

Código HTML com JavaScript

```
<!DOCTYPE html>
<html>
  <title>Título da Página</title>
  <body>
    texto do HTML antes do JS
    <p id="testeJS"> </p>
    <script
      charset="ISO-8859-1"
      type="text/javascript">
      document.getElementById("testeJS").innerHTML = "texto do JS";
    </script>
    texto do HTML depois do JS
  </body>
</html>
```

Resultado no Navegador

```
texto do HTML antes do JS  
texto do JS  
texto do HTML depois do JS
```



Você Sabia? (curiosidades)

Programação Orientada a Objetos é um paradigma de programação que usa abstração para criar modelos baseados no mundo real. E o método é uma ação do objeto, como ligar, desligar, frear se estivermos representando um veículo, por exemplo. É uma subrotina ou função associada a uma classe.

Fonte: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Introduction_to_Object-Oriented_JavaScript

4.4. Elementos Básicos de Programação em JavaScript

Agora a gente vai fazer uma rápida passagem por alguns conceitos fundamentais, que são utilizados em todas as linguagens de programação. Aproveite para fazer uma revisão ou aprender o básico.

4.4.1. Variáveis

São endereços de memória associados a um determinado nome (também chamado de identificadores), que guardam algum dado em tempo de execução. Você pode atribuir qualquer nome a uma variável, desde que siga as regras abaixo.

- Não utilize símbolos especiais. Por exemplo, ç ou acentos;

- O nome da variável deve começar sempre com uma letra. Por exemplo, usuario1;

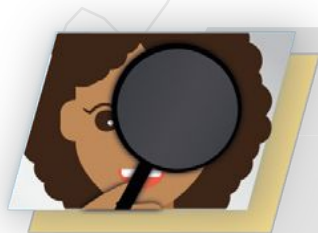
- Recomenda-se que os nomes de variáveis comecem sempre com letras minúsculas.

Observações

- O JavaScript é case sensitive, ou seja, considere letras maiúsculas e minúsculas diferentes;

- Quando você criar uma variável, poderá utilizar a palavra var antes do nome da variável. Se usar, a referida variável será considerada uma variável local e não global.

Variável com Letra Minúscula	Variável com Letra Maiúscula
<pre><!DOCTYPE html> <html> <title>Título da Página</title> <body> <script charset="ISO-8859-1" type="text/javascript"> var palavra = "Goku"; var PALAVRA = "Jiren"; alert(palavra); </script> </body> </html></pre>	<pre><!DOCTYPE html> <html> <title>Título da Página</title> <body> <script charset="ISO-8859-1" type="text/javascript"> var palavra = "Goku"; var PALAVRA = "Jiren"; alert(PALAVRA); </script> </body> </html></pre>
Resultado no Navegador	Resultado no Navegador
	



Você Sabia? (curiosidades)

A Rede Nacional de Ensino e Pesquisa (RNP) provê a integração global e a colaboração apoiada em Tecnologias de Informação e Comunicação para a geração do conhecimento e a excelência da educação e da pesquisa.

(Fonte: <https://www.rnp.br/institucional/quem-somos>)

A RNP disponibilizou uma série de videoaulas, entre as quais destaco a aula sobre “Tipos de Dados, Variáveis e Comandos de Entrada e Saída”. Caso você tenha ainda alguma dúvida sobre estes assuntos básicos, recomendo que você assista a essa videoaula.

Disponível em: http://videoaula.rnp.br/v.php?f=ufjf/ciencias_exatas/dcc119/aula1/dcc119_aula1.xml >. Acesso: 19 jan. 2018.



4.4.2. Operadores Aritméticos

São aqueles que permitem realizar algumas operações matemáticas básicas. Os operadores são os seguintes:

Símbolo	Operação Aritmética	Exemplo	Valor armazenado em x (após as instruções ao lado)
+	Soma	<code>var x = 3 + 2;</code>	5
-	Subtração	<code>var x = 3 - 2;</code>	1
*	Multiplicação	<code>var x = 3 * 2;</code>	6
/	Divisão	<code>var x = 3 / 2;</code>	1.5
%	Resto da divisão	<code>var x = 10 % 6;</code>	4
++	Incremento	<code>var x = 4; x++;</code>	5
--	Decremento	<code>var x = 4; x--;</code>	3

4.4.3. Uso do if e de operadores relacionais

Digamos que você entre com uma senha no Facebook. Se a senha estiver correta, abre a página do perfil. Caso contrário, mostra uma mensagem de erro. Como resolver isso em programação? Poderia usar um if.

A estrutura do if é utilizada para simular tomadas de decisão. Se um teste lógico for verdadeiro, executamos um bloco de instruções, caso contrário, executamos outro bloco de instruções, mas nunca os dois ao mesmo tempo.

Os testes são realizados utilizando operadores relacionais, a saber:

Símbolo	Descrição	Exemplo	Resposta do Teste
==	igual	<u>var</u> x = 3; <u>var</u> y = 4; if(x == y)	falso
!=	diferente	<u>var</u> x = 3; <u>var</u> y = 4; if(x != y)	verdadeiro
>	maior	<u>var</u> x = 3; <u>var</u> y = 4; if(x > y)	falso
<	menor	<u>var</u> x = 3; <u>var</u> y = 4; if(x < y)	verdadeiro
>=	maior ou igual	<u>var</u> x = 3; <u>var</u> y = 4; if(x >= y)	falso
<=	menor ou igual	<u>var</u> x = 3; <u>var</u> y = 4; if(x <= y)	verdadeiro

Estrutura básica:

```
if (teste)
{
    instruções
else
{
    instruções
}
```

Código HTML com JavaScript

```
<html>
  <head>
    <title>Goku</title>
  </head>
  <body>
    <p id="testeJS"></p>
    <script
      charset="ISO-8859-1"
      type="text/javascript">
      var idadeGoku = 40;
      if(idadeGoku == 44)
        document.getElementById("testeJS").innerHTML =
          "Goku tem 44 anos";
      else
        document.getElementById("testeJS").innerHTML =
          "mude o valor da variável idadeGoku para 44";
    </script>
  </body>
</html>
```

Resultado no Navegador

mude o valor da variável idadeGoku para 44

4.4.4. Uso de laços de repetição

Digamos que você queira repetir seu nome 10 vezes na página do navegador. O que faria? Que código utilizaria? Nem pense em usar `document.write()` 10 vezes (risos).

Nesse caso o melhor seria usar um laço de repetição.

Laços de repetição são estruturas utilizadas para repetir uma ou mais instruções. As duas principais estruturas de repetição são `while` e `for`.

As estruturas básicas desses laços são:

Usando while

```
ValorInicialDoContador  
while(TesteDeParada){  
    instruções  
    AlteraçãoDoValorDoContador  
}
```

Usando for

```
for(ValorInicialDoContador; TesteDeParada; AlteraçãoDoValorDoContador){  
    instruções  
}
```



Exemplos:

Usando while	Variável com Letra Maiúscula
<pre> <html> <head> <title>Jogo</title> </head> <body> <script charset="ISO-8859-1" type="text/javascript"> var jogo = "Counter-Strike"; var contador = 1; while(contador <= 10){ document.write(contador+ ") "+ jogo+ "
"); contador++; } </script> </body> </html> </pre>	<pre> <html> <head> <title>Jogo</title> </head> <body> <script charset="ISO-8859-1" type="text/javascript"> var jogo = "Counter-Strike"; for(var contador = 1; contador <= 10; contador++){ document.write(contador+ ") "+ jogo+ "
"); } </script> </body> </html> </pre>

Usando while Variável com Letra Maiúscula

```

<html>
  <head>
    <title>Jogo</title>
  </head>
  <body>
    <script charset="ISO-8859-1"
      type="text/javascript">

      var iogo = "Counter-Strike";
      var contador = 1;
      while(contador <= 10){
        document.write(
          contador+

          ") "+
          jogo+
          "<br>");

        contador++;
      }
    </script>
  </body>
</html>

```

```

<html>
  <head>
    <title>Jogo</title>
  </head>
  <body>
    <script charset="ISO-8859-1"
      type="text/javascript">

      var iogo = "Counter-Strike";
      for(var contador = 1;
        contador <= 10;
        contador++){
        document.write(
          contador+

          ") "+
          jogo+
          "<br>");
      }
    </script>
  </body>
</html>

```



Você Sabia? (curiosidades)

Para saber mais sobre laços de repetição assista aos vídeos da Khan Academy.

- Introdução aos laços de repetição While. Disponível em: <<https://pt.khanacademy.org/computing/computer-programming/programming/looping/p/intro-to-while-loops> >
- Laços For! Um novo tipo de Estrutura de Repetição. Disponível em: <<https://pt.khanacademy.org/computing/computer-programming/programming/looping/p/for-loops-a-new-kind-of-loop> >.

Assista! Vale a pena.

5. Criando as suas funções em JavaScript

Como dissemos anteriormente, uma função é basicamente um bloco de código que realiza uma tarefa específica. Para que esse bloco seja executado, a função precisa ser chamada.

Syntaxe básica:

```
function nomeDaFunção ( parâmetros
){ instruções
  return dado;
}
```

Onde:

- function é a palavra-chave que define uma função;
- nomeDaFunção é nome que você dá para a função, que será usado quando você for chamá-la;
- parâmetro é a informação que a função precisa saber para ser executada. Por exemplo, a função que calcula a raiz quadrada precisa saber de qual número será calculada a raiz quadrada.
- return é opcional e serve para retornar alguma informação para quem chama a função.

Exemplo:

Código HTML com JavaScript e com CSS

```
<html>

  <head>

    <title>Funções</title>
    <style type="text/css">

      .botao {
        font-weight: 900;

        background-color:red;

        color: white;

        padding: 10px 30px;

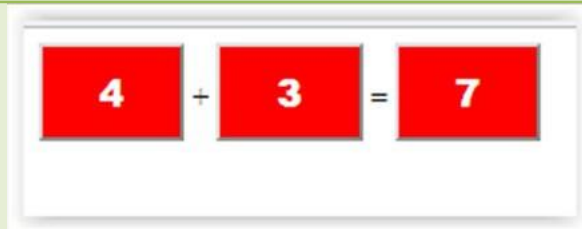
        text-align: center;

        font-size: 20px;
      }
    </style>
  </head>
  <body>

    <button class="botao" id="valor1" width="50" height="50"></button> +
    <button class="botao" id="valor2" width="50" height="50"></button> =
    <button class="botao" id="resultado" width="50" height="50"></button>
    <script charset="ISO-8859-1"
      type="text/javascript">
```

```
function soma(){  
    document.getElementById("valor1").innerHTML = "4";  
    document.getElementById("valor2").innerHTML = "3";  
    document.getElementById("resultado").innerHTML = "7";  
}  
soma();  
</script>  
</body>  
</html>
```

Resultado no Navegador



4.6. Eventos em JavaScript

Normalmente um programa é executado sequencialmente. E se você quisesse que todas as vezes que passasse o mouse sobre uma figura, aparecesse uma mensagem? Como você faria? Use um evento. Existem dezenas de eventos, apresentamos os eventos mais utilizados, a saber:

Evento	Descrição
onchange	Indica o que fazer quando um elemento do HTML mudar. Por exemplo, quando preencher o campo ou selecionar um item de um formulário.
onclick	Indica o que fazer quando clicar sobre um elemento do HTML.
onsubmit	Indica o que fazer quando clicar em botão submit de um formulário.
onmouseover	Indica o que fazer quando o mouse passar sobre um determinado elemento do HTML.
onload	Indica o que fazer quando o navegador termina de carregar uma página.
onunload	Indica o que fazer quando o navegador abandonar uma página HTML.

Exemplo:

Código HTML com JavaScript e com CSS

```

<html>
  <body>
    <h1>Clique sobre a imagem</h1>
    
    <script type="text/javascript">
      function mudaImagem() {
        var image = document.getElementById('dbz');
        if (image.src.match("saiyan")) {
          image.src =
"https://img00.deviantart.net/3cd7/i/2016/222/a/6/goku_by_bthrgking-
dadbnvv.jpg";
        } else {
          image.src =
"https://img00.deviantart.net/c429/i/2016/222/f/5/super_saiyan_3_goku_by_bthrg
king-dadbo7l.jpg";
        }
      }
    </script>
  </body>
</html>

```

Resultado no Navegador (imagem inicial)	Resultado no Navegador (depois do click)
Clique sobre a imagem 	Clique sobre a imagem 



Você Sabia? (curiosidades)

Existem muitos eventos diferentes e listar e explicar todos eles não seria muito produtivo. Então, à medida da sua necessidade, você pode pesquisar qual evento é mais adequado para resolver um problema específico.

Para ajudar na sua pesquisa, indico um site onde consta uma lista mais completa de eventos em JavaScript.

- Desenvolvimento de Eventos. Disponível em: <http://tdn.totvs.com/display/public/fluig/Desenvolvimento+de+Eventos>. Acesso: 19 jan. 2018.
- Referência do evento. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/Events>. Acesso: 19 jan. 2018.



Saiba mais!

Viu como eventos são uma ferramenta poderosa para o desenvolvedor web? Você conseguiu entender o conceito de evento? Caso reste alguma dúvida, proponho que você assista a mais uma videoaula da RNP.

No vídeo abaixo, é apresentado novamente o conceito de evento, os principais tipos e diz como associar trechos de código aos eventos e outros detalhes importantes.

Eventos.

Disponível em: <

[http://videoaula.rnp.br/v.php?f=/cederj/sistemas_comp/ead05002/](http://videoaula.rnp.br/v.php?f=/cederj/sistemas_comp/ead05002/Aula_017/Aula_017.xml)

[Aula_017/Aula_017.xml](http://videoaula.rnp.br/v.php?f=/cederj/sistemas_comp/ead05002/Aula_017/Aula_017.xml) >. Acesso: 19 jan. 2018.

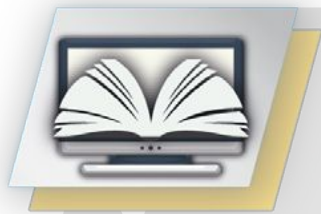


Bora rever!

Nesta lição a gente viu muita coisa sobre JavaScript. Vimos os principais editores para desenvolver seus projetos em JavaScript.

Aprendemos que

podemos inserir o JavaScript dentro do próprio HTML ou podemos armazená-lo em um arquivo externo. Conhecemos algumas formas de mostrar uma informação na página do navegador. Fizemos uma rápida revisão sobre os principais elementos de programação. Aprendemos como criar funções e como criar eventos no JavaScript.



Sites indicados

JavaScript Tutorial. Disponível em: <<https://www.w3schools.com/js/default.asp>>. Acesso em: 19 jan. 2018.

Qual é a diferença entre o JavaScript e o Java?. Disponível em: <https://www.java.com/pt_BR/download/faq/java_javascript.xml>. Acesso em: 19 jan. 2018.

Introdução ao JavaScript Orientado a Objeto. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Introduction_to_Object-Oriented_JavaScript>. Acesso em: 19 jan. 2018.

O que é uma variável?. Disponível em: <<https://youtu.be/-ZMCNZXmzZk>>. Acesso em: 19 jan. 2018.

Introdução aos laços de repetição While. Disponível em: <<https://pt.khanacademy.org/computing/computer-programming/programming/looping/p/intro-to-while-loops>>. Acesso em: 19 jan. 2018.

Laços For! Um novo tipo de Estrutura de Repetição. Disponível em: <<https://pt.khanacademy.org/computing/computer-programming/programming/looping/p/for-loops-a-new-kind-of-loop>>. Acesso em: 19 jan. 2018.

HTML DOM Events. Disponível em: <https://www.w3schools.com/jsref/dom_obj_event.asp>. Acesso em: 19 jan. 2018.



Glossário

- DOM

O Modelo de Objeto de Documento (DOM) é uma interface de programação para documentos HTML, XML e SVG. Ele fornece uma representação estruturada do documento como uma árvore. O DOM define métodos que permitem acesso à árvore, para que os mesmos métodos possam alterar a estrutura, estilo e conteúdo do documento. O DOM fornece uma representação do documento como um grupo estruturado de nós e objetos, possuindo várias propriedades e métodos. Os nós também podem ter manipuladores de eventos que lhe são inerentes e, uma vez que um evento é acionado, os manipuladores de eventos são executados. Essencialmente, ele conecta páginas web a scripts ou linguagens de programação.

Fonte: https://developer.mozilla.org/pt-BR/docs/DOM/Referencia_do_DOM

- JavaScript

JavaScript® (às vezes abreviado para JS) é uma linguagem leve, interpretada e baseada em objetos com funções de primeira classe, mais conhecida como a linguagem de script para páginas Web, mas usada também em vários outros ambientes sem browser como node.js, Apache CouchDB e Adobe Acrobat. É uma linguagem de script multiparadigma, baseada em protótipo que é dinâmico e suporta estilos de programação orientado a objetos, imperativo e funcional. Saiba mais sobre JavaScript.

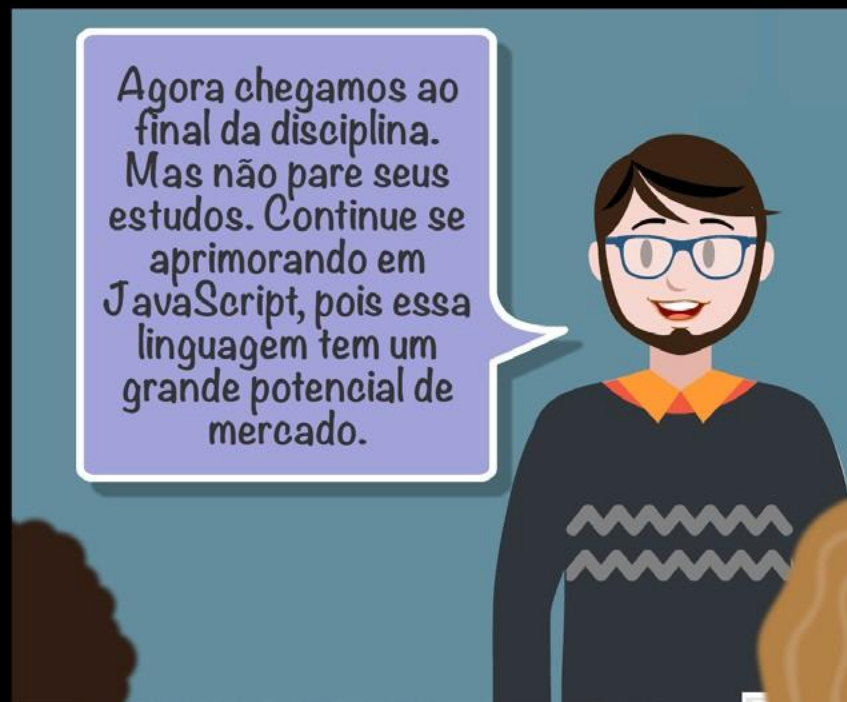
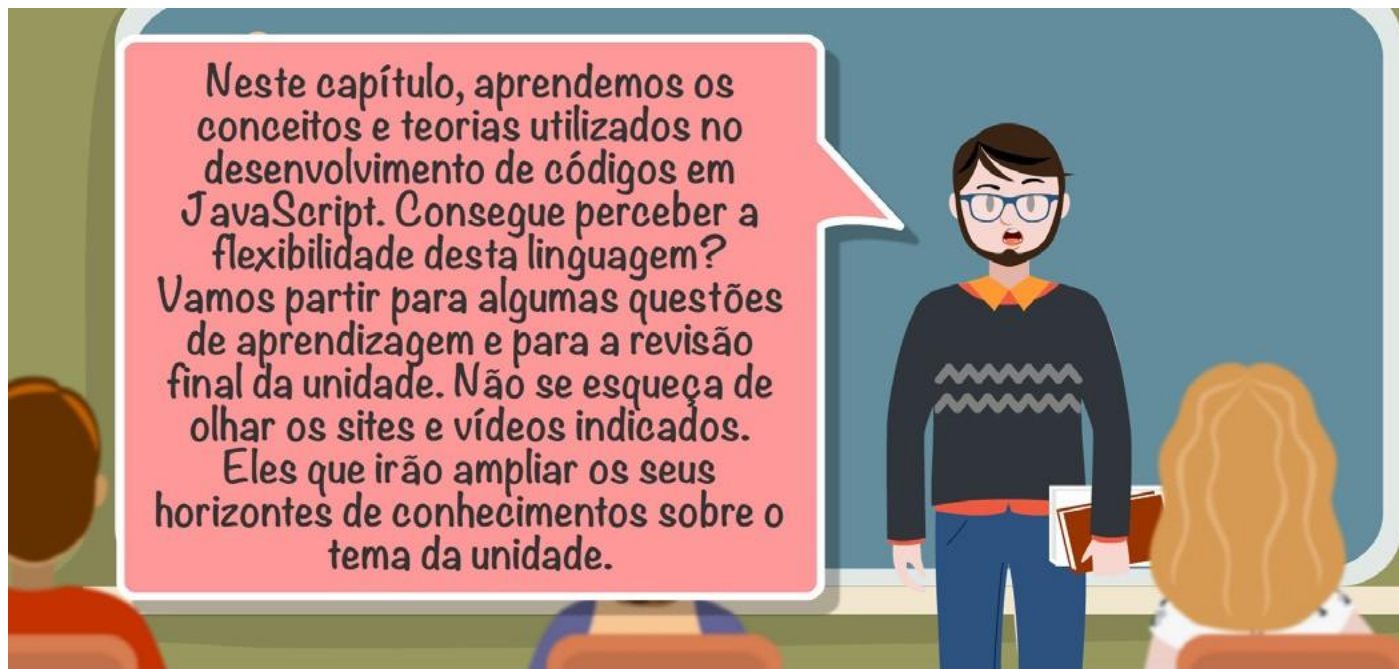
Fonte: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>

- Programação Orientada a Objetos

Programação Orientada a Objetos é um paradigma de programação que usa abstração para criar modelos baseados no mundo real. POO usa várias técnicas vindas de paradigmas previamente estabelecidos, incluindo modularidade, polimorfismo e encapsulamento.

Fonte:

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Introduction_to_Object-Oriented_JavaScript





Acesso aleatório - O acesso aleatório está diretamente relacionado à posição da memória endereçável. Dessa forma, dizemos que qualquer posição da memória pode ser selecionada, desde que endereçada corretamente.

Assembly - é uma notação legível por humanos para o código de máquina que uma arquitetura de computador específica usa, utilizada para programar códigos entendidos por dispositivos computacionais, como microprocessadores e microcontroladores. O código de máquina, que é um mero padrão de bits, torna-se legível pela substituição dos valores em bruto por símbolos chamados mnemônicos.

Chip - É uma pastilha de silício onde ficam os circuitos integrados compostos por milhões de componentes eletrônicos e essenciais aos computadores.

Dado - é tudo aquilo que sozinho, pode não significar nada. Por exemplo: 5 m, 10 kg, 20 km/h. Mas se dissermos “uma passarela tem 5m de altura”, isso pode se constituir numa informação para um motorista de caminhão, que evitaria trafegar naquela via sabendo que seu caminhão tem uma altura maior. Já a informação subentende dados organizados segundo uma orientação específica, para o atendimento ou emprego de uma pessoa ou grupo que os recebe.

Framework - Abstração que une códigos comuns entre vários projetos de software provendo uma funcionalidade genérica.

Hardware - Parte física do computador, componentes como placas de circuitos, discos de armazenamento, monitores e impressoras.

Informática - A informática é a disciplina que lida com o tratamento racional e sistemático da informação por meios automáticos e eletrônicos. Representa o tratamento automático da informação. Constitui o emprego da ciência da informação através do computador. Embora não se deva confundir informática com computadores, na verdade ela existe por que estes existem. Pode ser considerada como “informação automática”, ou seja, utilização de métodos e técnicas no tratamento automático da informação. Para tal, é preciso uma ferramenta adequada: o computador.

Interface - Parte do software que interage com o usuário para receber comandos e transmitir informações.

Kernel - Núcleo de um sistema operacional responsável por gerenciar os recursos do sistema computacional como um todo.

Loop - é um termo muito utilizado em informática, principalmente em redes e programação. Designa uma execução de uma tarefa que teoricamente pode nunca ter fim. Pode ser um aliado em processamentos que exigem volumes de dados muito grandes, como cálculos de previsão do tempo.

Memória RAM - Memória transitória. Funciona somente quando o computador está ligado. Quanto mais memória o pc tiver, mais programas e arquivos podem ser abertos simultaneamente.

Motherboard - Placa-mãe: placa principal que liga todos os componentes internos do computador.

Neologismo - emprego de palavras novas, derivadas ou formadas de outras já existentes, na mesma língua ou não.

Notebook - Computador portátil que traz como principal característica a integração e miniaturização da maior parte dos componentes, tornando-o leve e de pequenas dimensões.

Pixel - É a unidade de medida lógica utilizada para determinar a resolução de um monitor. Entretanto, a medida do pixel varia de acordo com a resolução física utilizada. Se for usada a resolução máxima do monitor, um pixel corresponde exatamente a um ponto no monitor. Caso a resolução adotada seja menor que aquela permitida pelo monitor, o pixel terá um tamanho maior de pontos, ou seja, um pixel será representado por mais de um ponto no monitor.

Processamento paralelo - é aquele processamento em que uma tarefa demasiadamente grande, que consome muito tempo de processamento, pode ser dividida em várias pequenas tarefas e ser

processada por dois ou mais processadores. Hoje em dia, com os computadores ligados em rede, é possível que esses processadores estejam em computadores separados, já que cada computador tem seu próprio processador.

Program Counter (PC) - Contador de Programas.

Semicondutores - são sólidos cujo material possui propriedades de condução ou isolamento da corrente, dependendo do tratamento químico que é dado a ele.

Software - Programa, conjunto de instruções lógicas que gerenciam as funções do computador e permitem a realização de tarefas específicas.

UC - Unidade de Controle.

ULA - Unidade Lógica e Aritmética.

Wikipédia - projeto de enciclopédia livre, baseado na web, colaborativo e apoiado pela organização sem fins lucrativos Wikimedia Foundation. Seus 19 milhões de artigos (691 696 em português) foram escritos de forma colaborativa por voluntários ao redor do mundo e quase todos os seus verbetes podem ser editados por qualquer pessoa com acesso ao site .

Referências

BROWN, Tim. Design Thinking. Uma Metodologia Poderosa para Decretar o Fim das Velhas Ideias. Alta Books, 2017.

FLANAGAN, David. JavaScript - O Guia Definitivo. Bookman, 2012.

FREEMAN, Eric. FREEMAN, Elisabeth. Use a Cabeça! HTML com CSS e XHTML. Alta Books, 2008.

MELO, Adriana. ABELHEIRA, Ricardo. Design Thinking and Thinking Design - Metodologia, Ferramentas e Uma Reflexão Sobre o Tema. Novatec, 2015.

MORRISON, Michael. Use a Cabeça! Javascript. Alta Books, 2008.

SHARP, Helen. ROGERS, Yvonne. PREECE, Jennifer. Design de Interação - Além da Interação Homem-computador, 3ª Ed., 2013.

SILVA, Maurício Samy. Fundamentos de Html5 e Css3. Novatec, 2015.

WATRALL, Ethan. Use a Cabeça! Web Design. Alta Books, 2009.

